DURGA

SYSTEM AND DEVELOPMENT MANUAL

Frode Randers

2026-07-06

Contents

1 Introduction

Durga is a BPMN-driven Kafka scaffolding tool. It reads BPMN models, extracts an executable subset of process semantics, and generates Java and Kafka-oriented project skeletons intended to be built, reviewed, deployed and operated as normal Kafka applications.

The project is a generator, not a workflow engine:

- BPMN models are parsed with Camunda (but not run in Camunda),
- worker, gateway, orchestrator and event handler classes are rendered with StringTemplate from the process graph,
- Kafka topics and reactive-messaging bindings are generated alongside the code,
- helper scripts and probes are generated so the resulting process can be exercised and observed,
- a separate Kafka Streams monitoring topology materializes process state and operational views.

This manual describes the current system. It focuses on what Durga generates, what runtime model those artifacts implement, and where the support boundary ends. It is written for two audiences: developers who own the generated process code, and platform or DevOps engineers who must provision Kafka topics, deploy workers, monitor process health, and decide which generated artifacts are acceptable for their production environment.

The generated output is not a managed workflow service. Durga supplies source code, topic manifests, helper scripts and monitoring projections; the operating team remains responsible for Kafka durability settings, credentials, deployment topology, alerting, backup/replay policy, and compatibility review before promoting a generated process.

The document is organized in that order. Section 2 describes the major subsystems. Section 4 explains the BPMN-to-Kafka execution mapping. Section 6 describes the canonical process-event and monitoring model. Section 10 describes the generated project output, and Section 28 summarizes maturity, constraints and current gaps.

2 System Overview

Durga has three parts:

- a BPMN parsing and generation pipeline,
- a generated Kafka-oriented process runtime,
- a monitoring and reporting path built around canonical lifecycle events.

2.1 Repository-level components

Scaffolder Parses a BPMN model and renders a generated project
(entry point: `org.gautelis.durga.tools.BpmnScaffolder`).
Generation support Model extraction and template coordination: `BpmnModelSupport`, `TaskRoutingGenerator`, `EventTemplateGenerator`, `SubProcessTemplateGenerator`, `GeneratedProjectSupport`.
Templates StringTemplate groups under `durga-tools/src/main/resources/templates-java/` define the generated Java classes, scripts, YAML, Maven build files and shared runtime helpers. A parallel `templates-rust/` directory holds the equivalent templates for the in-progress Rust code-generation target (see Section 18).
Monitoring runtime A Kafka Streams topology consumes canonical lifecycle events and materializes state, counts, latency summaries and stuck-instance views.

2.2 Shape of generated runtime

Each generated project contains:

- worker and routing services,
- a starter,
- a process orchestrator,
- helpers for user/manual task completion and failure testing,
- generated Kafka topic configuration,
- sample payloads and runnable scenario scripts.

Topics and handlers are deliberately visible so that BPMN concepts are projected concretely onto Kafka channels and lifecycle events.

2.3 Process self-registration

Each generated process publishes its BPMN 2.0 model to the `process-models` Kafka topic on startup via a `ModelRegistration` bean. The monitoring server discovers processes from this registry — no pre-configured process ID list is needed. The monitoring topology subscribes to all `process-events-*` topics via a regex pattern, making every process visible in a single monitoring instance.

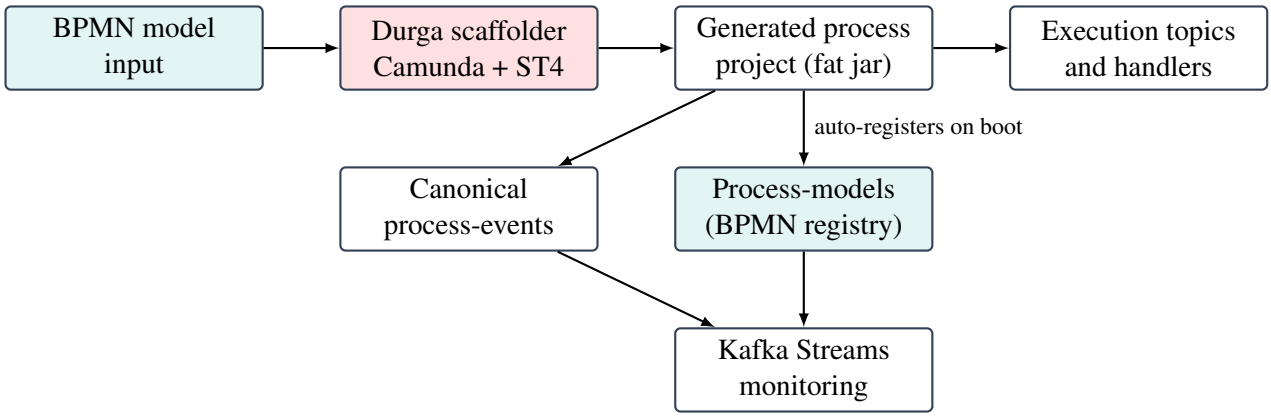

Figure 1: High-level flow: BPMN model → generated runtime → self-registration → monitoring

2.4 Canonical event model

The canonical lifecycle-event stream — by default a per-process topic `process-events-processId`, which the monitor consumes as the `process-events-*` family — is the observability backbone for latest state per instance, counts by current state, latency summaries, stuck-instance detection, dashboards and query endpoints. Execution topics may be useful for routing visibility, but the monitoring model depends on explicit projections rather than raw topic inspection.

3 Architecture Decisions

3.1 Canonical lifecycle stream

The canonical lifecycle-event stream — by default a per-process `process-events-`*`processId`* topic, consumed by the monitor as the `process-events-*` family — carries normalized lifecycle events. Execution topics carry work between generated handlers, but they are not the authoritative query source. State projections are built from lifecycle events because:

- execution topics represent transport history, not current truth,
- process instance state must be queryable independently of routing topology,
- counts, latency summaries and stuck-instance views are easier to derive from explicit lifecycle events than from task-topic inspection.

3.2 Visible generated runtime

Durga generates many explicit classes instead of a single hidden execution engine. The output is more verbose, but far easier to inspect — making BPMN-to-Kafka mapping concrete.

3.3 Generated helper scripts

Generated probes, scenario runners and manual interaction scripts let users exercise the runtime without writing custom producers and consumers.

3.4 Kafka Streams as the default read model

The monitoring side consumes one canonical stream and materializes queryable state. It solves the local architectural read-model problem directly rather than every production reporting problem.

3.5 Scope-aware subprocess handling

Subprocess support uses explicit entry and completion services because scope boundaries matter when cancellation, failure, escalation and event subprocesses are involved.

4 Execution Model

4.1 General mapping strategy

Durga maps a BPMN graph into a bounded Kafka runtime. In the current implementation, each supported BPMN element is converted into either:

- a generated service consuming from one topic and producing to one or more topics,
- a scope handler that turns subprocess semantics into lifecycle and routing events,
- or an external helper path that lets a human or another tool supply the missing runtime action.

The model is intentionally explicit. Instead of hiding process behavior behind one opaque runtime, the generated project exposes the process topology as concrete Kafka topics, generated services and helper scripts.

4.2 Core control-flow support

The executable subset currently includes:

- start events,
- service, user and manual tasks,
- exclusive and parallel gateways,
- call activities,
- explicit end-event completion,
- embedded subprocesses,
- several categories of boundary events,
- event subprocesses for selected start types.

The important distinction is that not all BPMN elements are treated the same way. A service task auto-completes in the generated worker model, while a user or manual task enters a waiting state and must be advanced through a generated completion helper.

4.3 Tasks and routing

Service tasks are generated as workers consuming `%processId%_%taskId%_input` and producing `%processId%_%taskId%_output`.

User and manual tasks consume their input topic, emit an `ACTIVITY_ENTERED` lifecycle event, and then wait for an external completion event generated via helper scripts.

Exclusive gateways are generated as routing services that evaluate simple conditions against the event payload and emit the selected next activity. Parallel splits fan out to several downstream inputs. Parallel joins use the generated process-state store to wait for the required set of incoming completions before forwarding the flow.

4.4 Boundary behavior

Boundary timer, error and escalation handlers are generated from BPMN boundary events and consume the canonical lifecycle stream on `process-events-monitor` (a logical channel mapped to the physical event topic configured via `-event-topic`). The current boundary implementation supports:

- interrupting and non-interrupting timer boundaries,
- interrupting and non-interrupting error boundaries,
- interrupting and non-interrupting escalation boundaries,
- task-attached and subprocess-attached variants.

Interrupting boundaries now have behavioral effect, not just monitoring effect. Generated handlers consult the shared cancellation registry so that canceled work is suppressed rather than continuing to emit normal downstream progress.

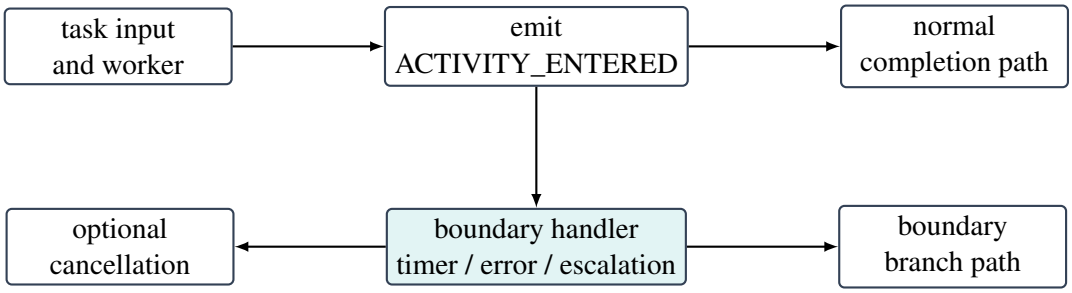

Figure 2: Generated boundary handling separates normal flow from boundary-triggered flow

4.5 Subprocesses and event subprocesses

Embedded subprocesses are generated with explicit entry and completion services. These services emit subprocess-level lifecycle events and route into and out of the internal subprocess graph. Nested subprocesses are supported with the same pattern.

Event subprocesses are currently implemented as scoped handlers rather than as a full general-purpose BPMN scope engine. The supported start types are:

- message,
- signal,
- timer inside an embedded subprocess scope,
- error inside an embedded subprocess scope,
- escalation inside an embedded subprocess scope.

Message and signal event subprocesses consume from dedicated external topics. Timer, error and escalation event subprocesses instead listen to scoped lifecycle signals on `process-events-monitor`. Interrupting event subprocesses emit cancellation for the enclosing scope before starting the event subprocess branch. Non-interrupting event subprocesses branch without canceling the parent flow.

4.6 Current execution boundary

The current execution boundary is broad enough for local process experimentation, but it is not full BPMN coverage. Complex gateways and more advanced scope semantics are still out of scope. BPMN data objects and data stores are collected as pipeline metadata and connector scaffolding inputs, but they are not executable flow nodes. That boundary should be treated as a deliberate implementation limit, not as an accidental omission.

5 BPMN Support And Limits

5.1 Well-supported areas

- start events and explicit end completion,
- service, user, manual, send, receive, script and business rule tasks,
- XOR, inclusive (OR) and parallel gateway routing,
- intermediate timer, message and signal events,
- task and subprocess boundary timer, error and escalation handling,
- call activities,
- embedded subprocesses including nested subprocesses,
- event subprocesses for message, signal, timer, error and escalation starts.

5.2 Current semantic boundaries

- cancellation propagation is pragmatic rather than globally token-precise,
- subprocess scope handling is explicit but lighter than a full BPMN execution scope manager,
- monitoring includes current-state indexes and coarse trend buckets, but is optimized for current-state inspection over long-term analytics,
- generated runtimes prioritize inspectability over minimalism,
- production readiness depends on the generated process implementation and deployment environment, not only on successful scaffolding.

5.3 Outside current scope

- full managed workflow-engine semantics,
- compensation and advanced BPMN exception orchestration,
- turnkey production operation without Kafka, Kubernetes, security and observability review,
- long-term analytics beyond the current monitoring projections.

See `doc/bpmn-kafka-coverage.md` for the per-element support matrix.

6 Monitoring and Observability

6.1 Canonical process-events topic

Durga emits normalized lifecycle events on the logical channel `process-events`. Generated workers, gateways, subprocess handlers, boundary handlers and helper publishers all emit events describing process progress, waiting states, completion, failure, escalation and cancellation. The default physical topic is `process-events-{processId}` so that each process definition is isolated at the transport level. The monitoring topology subscribes to `process-events-.*` to consume events from all processes.

This separates execution transport from queryable state. Execution topics remain useful for routing, but the monitoring model depends on projections built from explicit lifecycle events. For production use, treat the lifecycle topic as an operational contract: retention, access control, schema compatibility and replay policy determine how far back monitoring can recover and how safely downstream tools can consume process state.

6.2 Process self-registration

Processes self-register with the monitoring server by publishing their BPMN 2.0 model to the `process-models` Kafka topic. Each generated project includes a `ModelRegistration` CDI bean that runs at startup:

- reads the BPMN file from the classpath (`src/main/resources/{name}.bpmn`),
- publishes it to `process-models` with the process ID as the Kafka message key,
- logs success or failure without blocking the process startup.

The monitoring server caches models in the `process-models-store` state store.
The `/api/diagram?processId=`*id* endpoint serves the cached model. Model updates overwrite previous versions (latest-wins semantics).

This registry makes processes discoverable: `/api/processes/list` merges process IDs from both the count store (processes with live instances) and the model store (registered processes), so newly deployed processes appear in the dashboard before their first lifecycle event arrives.

6.3 Kafka Streams topology

The monitoring topology subscribes to `process-events-.*` via a regex pattern, monitoring all processes in a single instance. State stores use unsuffixed names. The SPA shows an inventory of all known processes with drill-down to per-process detail.

The topology consumes lifecycle events and materializes:

- latest process state by instance,
- counts by current state,
- latency summaries,
- stuck-instance views,
- trend buckets,
- BPMN model cache.

The main entry point is `org.gautelis.durga.monitoring.ProcessMonitoringApp`.

6.4 HTTP API

The monitoring service exposes endpoints for health, process discovery, instance lookup, counts, latency, stuck instances, alarm state, diagram serving, and a dashboard. All monitoring paths are served under the `/api/` prefix; a root `/health` endpoint is also available.

The local development loop is:

1. generate a process,
2. the process auto-registers its BPMN model on boot,
3. publish scenarios or task inputs,
4. observe lifecycle events,
5. query current state and latency,
6. view the BPMN diagram with activity overlays,
7. iterate on the BPMN model or generated runtime.

In production, the loop is stricter: monitor Streams state, query-store readiness, consumer lag on lifecycle and monitoring topics, stuck-instance counts, latency percentiles, DLQ volume and failed lifecycle events. The dashboard is an operator view over the Kafka Streams read model, not a replacement for Kafka broker, consumer group and infrastructure monitoring.

6.5 Fault detection

The fault detection topology consumes the same `process-events-.*` stream as the monitoring topology and evaluates each lifecycle event against a set of configured `AlarmConfig` rules. When a rule's criteria are met, an `AlarmEvent` is published to the `fault-alarms` topic for downstream notification, alerting, or dashboard integration.

6.5.1 Alarm layering

Alarms originate from three provenance layers, identified by `AlarmOrigin`. When multiple configs match the same scope the higher layer takes precedence.

Automatic Monitor-owned, always active, requiring no process configuration. Includes built-in stall and cascade detection (see STUCK / CASCADE below). Tunable via system properties (`durga.alarm.stuck.timeoutSeconds` etc.).
Opt-in A monitor-provided default that a process enables and may tune by including the alarm's properties in its BPMN model.
Explicit Fully specified by a process, typically through BPMN extension properties (`durga:alarm:<id>:<field>`) or authored directly in the Camunda Modeler plugin.

6.5.2 Alarm configuration

A single `AlarmConfig` record combines scope, a syndrome, severity, and a message template:

Scope Each config can target a specific process (`processId`), a specific activity (`activityId`), both, or neither (all events). A null `processId` matches every process; a null `activityId` matches process-level aggregates.
Event type Which `ProcessEvent.EventType` triggers evaluation (e.g. `PROCESS_FAILED`, `ACTIVITY_ESCALATED`). Optional for STUCK, CASCADE, and the SLA syndromes, which use a punctuator or default the event type from the scope.
Syndrome How observed behaviour is aggregated into an alarm (see below).
Severity `WARN` or `CRITICAL`.
Threshold Maximum count before alarm (required for COUNTED, SLIDING_WINDOW, CASCADE; ignored for HARD_ERROR and STUCK). For SLA_THROUGHPUT it is instead the *minimum* required completions per window.
Window duration Sliding-window size for SLIDING_WINDOW / CASCADE, idle-timeout for STUCK, the maximum allowed wall-clock latency for SLA_LATENCY, or the measurement period for SLA_THROUGHPUT.
Message A template rendered at alarm time; supports `${processId}`, `${activityId}`, `${count}`, `${processInstanceId}`, `${idleSeconds}` (STUCK), and `${latencyMs}` / `${limitMs}` / `${windowSeconds}` (SLA).
Origin One of `AUTOMATIC`, `OPT_IN`, or `EXPLICIT`.

6.5.3 Syndromes

6.5.3.1 Event-driven syndromes

The three event-driven syndromes are evaluated for each incoming lifecycle event.

HARD_ERROR First occurrence triggers alarm immediately. No threshold—every matching event produces an alarm. Suitable for critical failures that must never happen (e.g. schema validation failure on a compliance-critical field).
COUNTED Counts occurrences per (config, processId, activityId). The alarm fires once when the accumulated count exceeds `threshold`, then suppresses until the count drops back below the threshold (re-arm). Suitable for transient errors that become significant in volume (e.g. 5 task failures per process).
SLIDING_WINDOW Counts occurrences within a sliding time window. Old occurrences leak out after `windowDuration`. The alarm fires once when the windowed count exceeds `threshold`, then suppresses until the count drops back below threshold (re-arm). Suitable for burst detection (e.g. more than 3 `PROCESS_FAILED` events in a 60-second window).

6.5.3.2 Absent-progress syndromes

The two absent-progress syndromes react to the *absence* of lifecycle events rather than events that arrive. They are evaluated by a wall-clock punctuator scanning the active-instance index, not by the event-driven loop.

STUCK An active instance that has emitted no lifecycle event for longer than `windowDuration` (the idle-timeout) is reported as stuck. The alarm fires once per instance; subsequent punctuation cycles suppress until a fresh lifecycle event re-arms tracking. The template variable `${idleSeconds}` reports how long the instance has been idle.
CASCADE Collapses a surge of STUCK alarms into a single higher-severity alert. When the number of instances that newly became stuck within a rolling `windowDuration` exceeds `threshold`, one CAS-

CADE alarm fires. Suppressed until the window count falls back below threshold (re-arm). A CASCADE alarm's `processId` is `*` (system-wide); per-process CASCADE configs carry the actual process identifier.

6.5.3.3 Performance SLA syndromes

The SLA syndromes let a BPMN model declare a performance agreement — expressed as wall-clock latency or as throughput — for a single task or for the whole process, and alarm when observed performance violates it. Both use the same placement scoping as other alarms: activity-level properties measure that task, process-level aggregate (`$`) properties measure the whole process end to end, and process-level inherited (`*`) properties apply the same task-level SLA to every activity.

SLA_LATENCY A wall-clock ceiling. The maximum allowed duration is carried in `windowDuration`. For a task, latency is measured from `ACTIVITY_ENTERED` to `ACTIVITY_COMPLETED`; for the whole process it is measured from `PROCESS_STARTED` to `PROCESS_COMPLETED`. It is evaluated per completing instance, so each breaching completion produces one alarm. The template variables `${latencyMs}` and `${limitMs}` report the observed duration and the agreed limit. Authored with `durga:alarm:<id>:maxLaten`

SLA_THROUGHPUT A minimum-rate floor. `threshold` is the minimum number of completions required within each trailing `windowDuration` (the measurement period). It is scored by the wall-clock punctuator once a full window has elapsed since the first observation, fires once when the windowed count falls below the minimum, and re-arms when throughput recovers. The template variables `${count}`, `${threshold}`, and `${windowSeconds}` report the observed rate, the required minimum, and the window. Authored with `durga:alarm:<id>:threshold` and `:windowSeconds`.

6.5.4 Alarm emission

Triggered alarms are serialized as JSON and published to the `fault-alarms` topic. The topic is both an integration point for external notification systems and the input to Durga's alarm-state read model. Each `AlarmEvent` carries:

- `alarmId` — unique per firing,
- `configId` — back-reference to the triggering config,
- `syndrome`, `severity`, `message` — from config,
- `processId`, `processInstanceId`, `activityId` — scope,
- `triggerEventType`, `triggerTimestamp` — what fired it (`triggerEventType` is `null` for CASCADE alarms),
- `count` — current accumulated count at firing,
- `threshold` — config threshold (0 for HARD_ERROR, timeout-seconds for STUCK).

6.5.5 Alarm state read model

The monitoring server also starts an alarm-state topology that consumes `fault-alarms` and materializes one `AlarmStateView` per (processId, activityId, configId) scope in `alarm-state-store`. Process-wide alarms use `*` as the activity key component. Repeated alarm firings are folded into the same view:

- `status` is currently `ACTIVE`;
- `severity` keeps the strongest observed severity;

- `firstTriggeredAt` and `lastTriggeredAt` delimit the firing range;
- `fireCount` counts folded alarm firings;
- `lastProcessInstanceId`, `lastCount`, `lastAlarmId`, and `lastMessage` describe the newest firing.

The REST API exposes this read model through `/api/alarms`, `/api/processes/{processId}/alarms`, and `/api/instances/{processInstanceId}/alarms`. The dashboard uses these endpoints for the alarm KPI, process inventory alarm counts, alarm-state table, instance alarm details, and BPMN diagram alarm overlays. System-wide alarms (`processId: *`) are excluded from the process inventory and rendered as `system-wide` in the Alarm State widget.

6.5.6 State stores

The fault detection topology maintains six persistent state stores:

- `fault-config-store` — serialized `AlarmConfig` per processId, updated when a process model is published to `process-models`.
- `fault-counts-store` — current count per (config, processId, activityId).
- `fault-windows-store` — list of epoch-millis timestamps per scope (SLIDING_WINDOW, CASCADE, and SLA_THROUGHPUT windows).
- `fault-last-alarm-store` — count at which the last alarm fired, used for single-fire suppression.
- `fault-seen-store` — per-instance record of the last lifecycle activity observed (processId, activityId, last-seen epoch millis, alarm flag), used by the STUCK punctuator.
- `fault-latency-entry-store` — per (instance, scope) entry timestamp, used to measure SLA_LATENCY on completion; entries are cleared when the unit of work finishes.

6.5.7 Example configuration

```
AlarmConfig hard = new AlarmConfig("schema-fail-critical", null,
    "validate_high_value", ProcessEvent.EventType.ACTIVITY_ESCALATED,
    AlarmSyndrome.HARD_ERROR, 0, null,
    AlarmSeverity.CRITICAL, "Schema validation escalated",
    AlarmOrigin.EXPLICIT);

AlarmConfig counted = new AlarmConfig("proc-fail-warn", "order-proc",
    null, ProcessEvent.EventType.PROCESS_FAILED,
    AlarmSyndrome.COUNTED, 5, null,
    AlarmSeverity.WARN, "${count} failures in ${processId}",
    AlarmOrigin.EXPLICIT);

AlarmConfig window = new AlarmConfig("burst-detect", null, null,
    ProcessEvent.EventType.PROCESS_FAILED,
    AlarmSyndrome.SLIDING_WINDOW, 3, Duration.ofMinutes(1),
    AlarmSeverity.CRITICAL, "Burst: ${count} failures in 1 min",
    AlarmOrigin.EXPLICIT);
```

```
AlarmConfig stuck = new AlarmConfig("builtin:stuck", null, null, null,
    AlarmSyndrome.STUCK, 0, Duration.ofSeconds(120),
    AlarmSeverity.WARN, "Stuck ${processInstanceId} in ${activityId} "
        + "for ${idleSeconds}s",
    AlarmOrigin.AUTOMATIC);

AlarmConfig cascade = new AlarmConfig("builtin:cascade", null, null, null,
    AlarmSyndrome.CASCADE, 5, Duration.ofSeconds(60),
    AlarmSeverity.CRITICAL,
    "Cascade: ${count} instances stalled within 60s (threshold ${threshold})",
    AlarmOrigin.AUTOMATIC);

// SLA: a single task must complete within 2 seconds
AlarmConfig taskSla = new AlarmConfig("order-proc:score-sla", "order-proc",
    "score_risk", null,
    AlarmSyndrome.SLA_LATENCY, 0, Duration.ofMillis(2000),
    AlarmSeverity.WARN, "${activityId} took ${latencyMs}ms (limit ${limitMs}ms)",
    AlarmOrigin.EXPLICIT);

// SLA: the pipeline must complete at least 100 instances per minute
AlarmConfig rateSla = new AlarmConfig("pipeline:min-rate", "pipeline", null,
    null, AlarmSyndrome.SLA_THROUGHPUT, 100, Duration.ofSeconds(60),
    AlarmSeverity.CRITICAL,
    "Throughput ${count}/${windowSeconds}s below minimum ${threshold}",
    AlarmOrigin.EXPLICIT);
```

Because SLA configurations are usually part of the process contract, they are normally authored in the BPMN model as Camunda extension properties (optionally via the Durga modeler plugin) rather than in code. The monitor is handed the model when the process registers on `process-models` and extracts them automatically. For example, a per-task latency SLA and a process-wide throughput SLA:

```
<!-- on a ServiceTask: this task must finish within 2s -->
<camunda:property name="durga:alarm:sla:syndrome"     value="SLA_LATENCY"/>
<camunda:property name="durga:alarm:sla:maxLatencyMs" value="2000"/>
<camunda:property name="durga:alarm:sla:severity"     value="WARN"/>
<camunda:property name="durga:alarm:sla:message"
    value="${activityId} slow: ${latencyMs}ms > ${limitMs}ms"/>

<!-- on the Process (aggregate $ scope): >= 100 completions/min -->
<camunda:property name="durga:alarm:$rate:syndrome"      value="SLA_THROUGHPUT"/>
<camunda:property name="durga:alarm:$rate:threshold"     value="100"/>
<camunda:property name="durga:alarm:$rate:windowSeconds" value="60"/>
<camunda:property name="durga:alarm:$rate:severity"      value="CRITICAL"/>
<camunda:property name="durga:alarm:$rate:message"
    value="throughput ${count} below ${threshold}/${windowSeconds}s"/>
```

6.5.8 Integration with the monitoring server

The monitoring server starts the fault detection topology alongside the monitoring read-model topology as a separate `KafkaStreams` instance with its own application ID and state directory. Both subscribe to `process-events-.*`; the fault detector also consumes `process-models` so BPMN alarm properties can update runtime alarm configuration. A third `KafkaStreams` instance maintains the alarm-state read model from `fault-alarms`, and a fourth drives the validation (shadow-run) topology. Health output reports all four stream states: core monitoring, fault detection, alarm state, and validation.

Unit tests are in `FaultDetectionTest`, `BpmnAlarmConfigParserTest`, `AlarmStateViewTest`, `AlarmStateTopologyTest`, and `PluginResultTest`; see Appendix 1 for the test catalogue.

7 Monitoring Topology

7.1 Purpose

The monitoring subsystem is a separate runtime concern from the generated process execution graph. Generated handlers emit canonical lifecycle events; the monitoring topology consumes those events and turns them into queryable operational views.

Execution topics show how work is routed. The monitoring topology answers:

- where is a specific process instance now,
- how many instances are in a given state,
- which activities are taking the longest,
- which instances appear stuck,
- what are the coarse trend counts for starts, completions, and failures,
- which configured alarms are active and where they last fired.

7.2 Topology structure

The entry point is `ProcessMonitoringTopology.buildTopology()`. It subscribes to `process-events-.*` via regex, monitoring all processes in a single instance. It consumes events, filters null keys and values, and forks the stream into five parallel sub-topologies plus a BPMN model cache, each with a dedicated local state store:

Instance state Aggregates events into `ProcessStateView` per instance. Materialized in `process-state-store`, output to `process-state`. A filtered subset (active instances) is output to `process-active-state`.
State counts Regroups state views by `processId:state` key and counts instances. Materialized in `process-state-counts-store`, output to `process-state-counts`.
Activity latency Feeds events through a custom `ActivityLatencyProcessor` that records `ACTIVITY_ENTERED` timestamps in a persistent key-value store (`process-activity-entry-store`) and, on later completion or termination events, emits duration deltas. These deltas are aggregated into rolling `ActivityLatencyStats` per activity, then mapped to `ActivityLatencySummary` and output to `process-latency`.
Trends Filters to `PROCESS_STARTED`, `PROCESS_COMPLETED`, and `PROCESS_FAILED` events, maps them to hourly trend bucket keys, counts per bucket, and outputs `ProcessTrendPoint` records to `process-trends`. Materialized in `process-trends-store`.
Queryable replicas Six `builder.globalTable()` calls consume the output topics above plus `process-models`, each materialized as a global table with a `-global-store` or `-store` suffix. These are full replicas on every monitoring instance and are what the query service reads.
BPMN model cache A global table on `process-models` stores the latest BPMN 2.0 XML per process ID (String key, String value). Processes publish their models here on startup via `BpmnModelPublisher` or the generated `ModelRegistration` bean. The diagram endpoint serves models from this cache.

Fault detection and alarm state are deliberately separate stream topologies. `FaultDetectionTopology` consumes `process-events-.*` and `process-models`, evaluates BPMN alarm configuration, and emits `AlarmEvent` records to `fault-alarms`. `AlarmStateTopology` then consumes `fault-alarms` and folds repeated firings into `AlarmStateView` records in `alarm-state-store`. The monitoring

application runs these as separate KafkaStreams instances with separate application IDs and state directories.

Validation mode adds a fourth separate topology. ValidationTopology merges candidate task outputs (validation-candidate-outputs) with the prior/production output (the ACTIVITY_COMPLETED lifecycle event), keyed on processId:activityId:processInstanceId, and materializes a ValidationResult per task per instance in validation-results-store. It runs as its own KafkaStreams instance and is described in Section 9.

7.3 State stores

The topology maintains eleven state stores:

Store	Type	Purpose
process-state-store	Local KTable aggregate	Per-instance latest ProcessStateView
process-state-counts-store	Local KTable count	Counts per processId:state key
process-latency-store	Local KTable aggregate	Rolling ActivityLatencyStats per activity
process-activity-entry-store	Persistent KV	Timestamps for the latency temporal join
process-trends-store	Local KTable count	Trend bucket counts
process-models-store	Global table	BPMN 2.0 XML per process ID (process registry)
*-global-store (5×)	Global table	Full replicas for HTTP/CLI query access

The five local stores drive computation — aggregation, counting, and temporal joins. Six global tables serve read access: five (*-global-store) hold full replicas of the computed output topics, and process-models-store holds the BPMN model registry. The output topics (process-state, process-state-counts, process-active-state, process-latency, process-trends) decouple computation from query access: any monitoring instance can spin up and receive a full read replica without recomputing.

The fault and alarm-state stores are separate from the core process read model:

- fault-counts-store, fault-windows-store, fault-last-alarm-store, fault-config-st
  fault-seen-store, and fault-latency-entry-store support the fault-detection and performance-SLA syndromes and alarm suppression.
- alarm-state-store is the queryable active-alarm read model consumed by the HTTP API and dashboard.

7.4 Latency computation

Activity latency uses a temporal join pattern without windowed stream-stream joins. The ActivityLatencyProcessor stores each ACTIVITY_ENTERED event's timestamp under the key

instanceId:activityId in the persistent process-activity-entry-store. When a later ACTIVITY_COMPLETED, ACTIVITY_ESCALATED, ACTIVITY_CANCELLED, or PROCESS_FAILED event arrives for the same key, the processor computes the duration and emits an ActivityLatencyDelta.

These deltas are grouped by a canonical activity key (processId, version, activityId, event type) and aggregated into ActivityLatencyStats — a running count, sum, min, and max — then mapped to ActivityLatencySummary for output.

7.5 Processing guarantees

The topology is configured with AT_LEAST_ONCE semantics. This trades exactly-once guarantees for higher throughput and lower latency. State views may transiently reflect duplicate or stale updates, which is acceptable for a monitoring read model. The state directory defaults to target/kafka-streams-state and can be overridden via the durga.streams.state.dir system property.

7.6 Query access pattern

The process query service (ProcessMonitoringQueryService) accesses only the global table stores via KafkaStreams.store(). It uses point lookups for instance state and full-store scans for counts, latency, trends, and stuck detection — filtering in memory by process ID where needed. No interactive queries or cross-instance RPC is required, since global tables replicate the entire content to every monitoring instance.

The alarm query service (AlarmStateQueryService) reads alarm-state-store from the alarm-state KafkaStreams instance and scans it for global, process-specific, and latest-instance-specific alarm views.

7.7 Relationship to demos

Repository-level demo publishers and scenario runners emit synthetic lifecycle events directly to process-events, making the monitoring subsystem independently testable without a full generated process runtime.

7.8 Current shape

The monitoring path now supports:

- process self-registration: generated projects publish their BPMN models on startup,
- process discovery from the model cache, so new processes appear before events flow,
- queryable state replicated from monitoring topics into global read-side stores,
- latency summaries maintained incrementally via a custom processor,
- active-instance queries using a dedicated active-state index,
- coarse history as bucketed trend counts for process starts, completions, and failures,
- per-process BPMN diagrams served from the model cache with activity overlays,
- fault detection from BPMN alarm configuration,
- queryable active-alarm state derived from fault-alarms,

- per-task validation comparisons (candidate vs. prior output) derived from `validation-candidate-outputs`, materialized in `validation-results-store`.

History is intentionally coarse — Durga supports basic trend reporting but is optimized more for current-state inspection than long-retention analytical reporting.

8 Monitoring Interfaces

The monitoring subsystem exposes these interfaces over Kafka Streams read models:

- a Quarkus HTTP API and embedded SPA,
- a command-line client.

8.1 HTTP service

Entry point: `ProcessMonitoringApp` (Quarkus). Query layers: `ProcessMonitoringQueryService` for process state and `AlarmStateQueryService` for alarm state.

Endpoints (all under `/api/`):

- `/health` — Kafka Streams readiness for probes and load balancers,
- `/api/health` — API-namespace health alias,
- `/api/processes/list` — all known process IDs (from counts + model registry),
- `/api/instances/{instanceId}` — latest instance view,
- `/api/processes/{processId}/counts` — process-specific state counts,
- `/api/processes/{processId}/latency` — activity latency summaries,
- `/api/processes/{processId}/trends` — bucketed start/completion/failure history,
- `/api/counts` — global state-count view,
- `/api/alarms` — global active alarm-state view,
- `/api/processes/{processId}/alarms` — process-specific active alarm-state view,
- `/api/instances/{instanceId}/alarms` — active alarms whose latest firing belongs to the instance,
- `/api/stuck` — stuck-instance detection,
- `/api/validation/summary?processId=`*id* — per-task validation outcome counts; `processId` is optional (omit it for all processes) (see Section 9),
- `/api/validation/results?processId=&taskId=&status=` — individual validation comparisons,
- `/api/validation/instances/{instanceId}` — validation comparisons for one instance,
- `/api/diagram?processId=`*id* — BPMN 2.0 XML from the model cache,
- `/api/metrics` — Prometheus-format metrics (activity latency and samples, SLA-latency violations, active alarms and SLA observed/limit gauges, streams state, and validation counts).

The diagram endpoint serves models from the `process-models-store` Kafka state store, where processes self-register by publishing their BPMN XML on startup.

The HTTP API is designed for internal operational use. Put it behind the same network and identity controls used for other production admin interfaces before exposing it outside a trusted environment.

8.2 CLI client

The `ProcessMonitoringClient` takes a base URL followed by a command:

```
java -cp durga-monitor/target/durga-monitor-0.1.0-beta.1-runner.jar \
  org.gautelis.durga.monitoring.ProcessMonitoringClient \
  http://localhost:8081 <command>
```

Supported commands:

```
health
instance <processInstanceId>
counts [processId]
latency <processId>
trends <processId>
stuck [processId] [olderThanSeconds]
```

The base URL defaults to `http://localhost:8081`. Set `DURGA_MONITORING_API_KEY` in the environment to send an `Authorization: Bearer` header when the API requires authentication.

8.3 Dashboard

The Svelte SPA is built from `monitoring-ui/` and served directly from the monitoring backend. It shows:

- a process inventory with all known process IDs and aggregate KPIs,
- alarm KPIs and process-level alarm counts,
- per-process drill-down: state mix, lifecycle trends, activity latency, stuck instances, and alarm state,
- a validation report: per-task outcome counts and per-instance input/prior/candidate diffs when candidate tasks run in validation mode (see Section 9),
- instance detail lookup with latest alarm details,
- a BPMN diagram viewer with color-coded activity overlays (alarms, latency, and state).

The diagram viewer uses `bpmn-js` with built-in zoom (scroll wheel), pan (grab tool), and fit-to-viewport controls.

The monitoring SPA is served from the same Quarkus HTTP server that hosts the REST API. It polls `/api/*` endpoints and renders an interactive dashboard with a BPMN diagram viewer.

8.4 How the interfaces fit together

The interfaces consume the same materialized read models:

1. processes self-register by publishing BPMN models to `process-models`,
2. lifecycle events flow through `process-events-*`,
3. the fault detector emits matching `AlarmEvent` records to `fault-alarms`,
4. Kafka Streams materializes state, counts, latency, trends, the model cache, and alarm state,
5. the HTTP layer exposes that materialized state,
6. the CLI and dashboard consume the same HTTP API.

The operator workflow is:

- deploy a process — it auto-registers its model on boot,
- the dashboard shows the new process in the inventory,
- publish a scenario or task outcome,
- watch lifecycle events,
- query one instance or inspect counts, latency, and alarms,
- view the BPMN diagram with live overlays.

9 Validation Mode

9.1 Purpose

Validation mode runs a not-yet-released implementation of a process task against real input, alongside the current (or prior) version, and reports where the new version produces the same output and where it diverges. It answers a concrete release question: *if I deploy this candidate task, what changes downstream?* Sometimes an identical input→output is expected (a refactor); sometimes divergence is intended (a functional correction). Validation mode does not judge — it produces the evidence and lets a human decide.

9.2 Per-task isolation

Validation is handled *per task*. Each candidate task is fed the production input *for that task* — the prior version's output of its predecessor — not the candidate's own reworked output. Candidate outputs are therefore never chained together. A divergence introduced by a modified early task cannot cascade into and contaminate the comparison of later tasks: every task is compared in isolation against production for the identical shared input.

9.3 Shadow worker and the dedicated index

For each plugin task, scaffolding with `-validation` generates an additional *shadow worker* beside the production worker. The shadow worker:

- consumes the same production input topic through a *dedicated consumer group* (`%processId%_%taskId%_validation`), so the production consumer offset — the current input index — is never advanced or disturbed;
- starts at `latest` by default (live-concurrent shadowing) and can be pointed at recent history via `DURGA_VALIDATION_OFFSET_RESET` (a bounded replay near the tail — not from the very beginning, since old input may not be representative);
- runs the candidate implementation with side effects suppressed;
- writes nothing to the task output topic, `process-events`, or the dead-letter queue. Its output is diverted to the shared `validation-candidate-outputs` topic as a `ValidationCandidateOutput` record carrying the shared input, the candidate output, disposition, idempotency key, and any error.

Live-concurrent and historic replay differ only in where the dedicated consumer starts; the matching mechanism is identical in both.

9.4 Side-effect suppression

Because the shadow worker never publishes to the task output topic, `process-events`, or the DLQ, downstream tasks never observe a candidate result. For handle-aware (`materialize`) execution, `PluginExecutionSupport.executeSandboxed` runs the transform but writes no object to the store: it returns a synthetic handle whose content hash still reflects the real output, so the meaningful part stays comparable while volatile fields (uri, timestamps) are excluded by comparison ignore paths. Version 1 targets pure transform (plugin) tasks; tasks that perform their own external effects (send tasks, external calls, object-store writes inside a plugin body) are out of scope for validation and must not be shadowed.

9.5 Comparison and report

The monitor's `ValidationTopology` pairs each candidate output against the prior/production output for the same input — the `ACTIVITY_COMPLETED` lifecycle event — keyed identically on `processId:activityId:processInstanceId`. The two sides are merged into a single aggregate so the comparison is robust to arrival order: in live mode the candidate may arrive before or after the production output. A candidate seen before any prior output yields `PRIOR_MISSING`, superseded when the prior output later arrives.

Comparison is a structural, order-sensitive JSON diff (`JsonComparison`) with configurable *ignore paths* (dot-notation, with `*` and `[*]` wildcards; set via `durga.validation.ignore.paths`) so volatile fields such as generated timestamps or identifiers do not register as divergence. Each comparison produces a `ValidationResult` with one of four outcomes:

`EQUAL` candidate output identical to prior after ignore paths.
`DIFF` candidate output differs; the located differences (path, kind, prior value, candidate value) are attached.
`PRIOR_MISSING` no production output found for the shared input.
`CANDIDATE_ERROR` the candidate failed or requested a non-success error strategy.

Results are written to `validation-results` and materialized in the `validation-results-store` global table for query.

9.6 Topics and naming

- `%processId%_%taskId%_validation` — dedicated consumer group for the shadow worker (reads the production input topic).
- `validation-candidate-outputs` — shared topic carrying candidate outputs from every shadow worker (Java and Rust).
- `validation-results` — comparison results, materialized in `validation-results-store`.

9.7 Reporting interfaces

The monitor exposes the report under `/api/validation/`:

- `/api/validation/summary?processId=`*id* — per-task counts (equal / diff / prior-missing / candidate-error / total); `processId` is optional (omit it for all processes),
- `/api/validation/results?processId=&taskId=&status=` — individual comparisons, optionally filtered,
- `/api/validation/instances/{instanceId}` — all validated tasks for one instance.

The dashboard adds a *Validation Report* panel to the per-process drill-down: a per-task summary table and, per instance, an expandable comparison showing the shared input, prior output, candidate output, and a highlighted diff table. Prometheus metrics `durga_validation_total`, `durga_validation_diff`, and `durga_validation_candidate_error` are exported per `process_id,task_id` on `/api/metrics`.

9.8 Java and Rust parity

Both code-generation targets emit shadow workers. The `ValidationCandidateOutput` wire record is byte-compatible across targets (camelCase fields matching the Java record), so a mixed Java/Rust fleet feeds the same `validation-candidate-outputs` topic and the same monitor comparator. The Rust runtime crate (`durga-rust`) provides the matching `ValidationCandidateOutput` type and a `plan_validation_output` function; generated Rust projects add a `%taskId%_validation` binary per plugin task using `run_validation_worker` with the same dedicated-group and offset-reset behaviour.

9.9 Comparison model summary

Type	Role
`ValidationCandidateOutput`	Shadow worker → comparator wire record: shared input, candidate output, disposition, error.
`JsonComparison`	Structural JSON diff with ignore-path normalization.
`ValidationResult`	Comparator → report record: outcome plus located differences and both payloads.
`ValidationSummary`	Per-task aggregate counts surfaced to the API, dashboard, and metrics.

10 Generated Projects

10.1 Generated files

The generated output contains:

- operational classes (workers, gateways, orchestrator) in the configured package,
- contract interfaces for custom activities (*Name*`Contract.java`),
- shared runtime contracts for data handles, Vannak metadata events, and handle-aware plugin execution,
- probe and exerciser classes (starter, publishers, watchers, scenario runner) in `<package>.probes`,
- a copy of the original BPMN model under `src/main/resources/`,
- a `ModelRegistration` CDI bean that publishes the BPMN model to the `process-models` Kafka topic on startup, enabling process self-registration with the monitoring server,
- a generated `pom.xml`,
- merged Kafka channel configuration in `application.yml`,
- a topic-creation script,
- sample payload data in `task-payloads.json`,
- a generated README,
- deployment and operations artifacts (`Dockerfile`, `k8s.yml`, `docker-compose.test.yml`, `alerts.yml`, `topics.yml`, `summary.json`), with additional artifacts emitted for specific flags (e.g. `keda-scalers.yml` and `acls.sh` for `-strimzi`/`-separate-workers`, and `connect-source.` for `-connect`),
- helper scripts for demos, inputs, completions, failures, escalations and observers.

When scaffolded with `-validation`, the output additionally contains a validation-mode shadow worker per plugin task (*Name*`ValidationWorker.java`) and the `ValidationCandidateOutput` runtime contract, with the dedicated validation consumer group and the `validation-candidate-outputs` channel merged into `application.yml` and the validation topics added to the topic-creation script (Section 9).

The BPMN model copy is kept alongside the source code and is the input to regeneration. During the build, `ModelEnricher` writes implementation metadata for custom activities back into this copy, keeping the model in sync with the implementation.

10.2 Contract interfaces and custom workers

When a `ServiceTask` is annotated with `camunda:plugin="custom"`, the scaffolder generates:

- *Name*`Contract.java` — an interface extending `Plugin` that defines the contract for the custom implementation.
- *Name*`WorkerService.java` — a CDI bean that injects the contract interface and delegates each incoming message to the implementation. Includes a null guard and dead-letter routing.

Custom workers use the same output contract as registered plugin executors: returned JSON objects become the next event payload; scalar, array, and non-JSON results are wrapped as `{ "_value": ... }`; and

a `null` result keeps the incoming payload unchanged. Both registered plugin executors and custom workers delegate through `PluginExecutionSupport`, so they can either pass `DataHandle` references through as ordinary plugin payloads (`handleMode=manual`) or materialize referenced object-store bytes before and after plugin execution (`handleMode=materialize`).

After successful plugin or custom execution, generated workers emit a Vannak-compatible `DataIndividualMetadataEvent` to the `vannak-metadata-events` Kafka topic. This keeps Durga's lifecycle `ProcessEvent` small while exposing item-level metadata for Vannak.

The developer writes a separate implementation class (*Name*`Impl.java`) that implements the contract interface and is annotated with `@ApplicationScoped` for CDI discovery. The implementation class is never touched by the scaffolder.

10.3 Helper scripts

Generated scripts include:

- `demo-scenario.sh`,
- `send-task-input.sh`,
- `complete-task.sh`,
- `fail-task.sh`,
- `escalate-task.sh`,
- `complete-call-activity.sh`,
- `send-message-event.sh`,
- `send-signal-event.sh`,
- `watch-process-events.sh`,
- `watch-task-output.sh`,
- `topics.sh` (topic creation),
- `run-local.sh` (local run helper),
- `replay.sh` (DLQ inspection/replay),
- `deploy.sh` (deployment helper).

Additional scripts are emitted for specific flags: `acls.sh` (with `-strimzi`) and `connect.sh` (with `-connect`).

10.4 Development workflow

1. Model the pipeline in Camunda Modeler, selecting plugins from the catalog for standard operations and marking custom activities with `plugin="custom"`.
2. Generate the project from the BPMN model with Durga.
3. For custom activities: implement the generated contract interface in a separate Java file. Compile and test with standard tools.
4. Run `ModelEnricher` during the build to write implementation metadata back into the local BPMN model copy.
5. Build the generated project, run a starter or scenario, and observe progress through `process-events`, `vannak-metadata-events`, and the monitoring views.

6. When the pipeline needs changes, edit the enriched BPMN model and regenerate. Custom implementations survive untouched; plugin workers are fully regenerated.

11 Deployment

Durga-generated projects are plain Java applications that connect to Kafka. They can be deployed as JARs, Docker containers, or Kubernetes workloads with no Durga-specific runtime service beyond the generated code and a Kafka cluster.

11.1 Architecture

Each generated process instance is a set of Quarkus-based workers (one per BPMN task) communicating through Kafka topics. There is no central orchestrator at runtime — workers consume from input topics and produce to output topics independently. This means:

- every worker is horizontally scalable via Kafka consumer groups,
- worker deployment is stateless; durable state lives in Kafka topics and Kafka Streams stores, while some generated handlers still keep transient in-memory coordination state,
- the generated shaded JAR is self-contained (all dependencies bundled).

11.2 Local development & testing

The recommended local loop uses Docker Compose for Kafka and direct JVM execution for the pipeline:

```
# Start Kafka
cd setup && docker compose up -d

# Build and run the pipeline
cd generated/<project>
START_KAFKA=false BOOTSTRAP=localhost:9094 ./run-local.sh
```

For iterative development, keep Kafka running and rebuild the pipeline:

```
mvn -q clean package -DskipTests
java -cp target/<project>-all.jar \
  org.gautelis.durga.generated.probes.<Process>Starter localhost:9094
```

To test with the monitoring dashboard:

1. Start Kafka as above.
2. Start the monitoring app:

   ```
   cd setup
   START_KAFKA=false ./demo-monitoring.sh
   ```

3. Run pipeline scenarios via the generated helper scripts.
4. Open `http://localhost:18081` for the Svelte dashboard or `http://localhost:18081/api` for the REST API.

11.3 Connection to Kafka

Kafka bootstrap servers are resolved slightly differently depending on the process, but all paths end at the same hardcoded default `localhost:9094`:

1. The monitoring app takes an optional first CLI argument, then falls back to the JVM system property `kafka.bootstrap.servers`, then the default.
2. Generated standalone (non-Quarkus) workers read the JVM system property `kafka.bootstrap.servers`, then the default.
3. Generated Quarkus workers read the environment variable `KAFKA_BOOTSTRAP_SERVERS` (via `application.yml`), then the default.

In containers the generated `Dockerfile` bridges the environment variable into the JVM property, so setting `KAFKA_BOOTSTRAP_SERVERS` works uniformly across worker styles (see below).

In Docker and Kubernetes, set `KAFKA_BOOTSTRAP_SERVERS` as a container environment variable. The generated `Dockerfile` passes it to the JVM:

```
ENTRYPOINT ["java", "-Xms64m", "-Xmx256m", \
  "-Dkafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:-localhost:9094}", \
  "-jar", "/app/pipeline.jar"]
```

11.4 Topic provisioning

Kafka topics must exist before the pipeline starts. There are three approaches, each supported by the generated artifacts.

11.4.1 Pre-creation via shell script

The generated `topics.sh` creates all required topics using the Kafka CLI tools bundled with the Kafka distribution:

```
KAFKA_BOOTSTRAP_SERVERS=kafka-broker:9092 ./topics.sh
```

This is suitable for development, staging, and controlled manual production setups. For production, review partition count, replication factor, cleanup policy and retention before running the script. Automated environments should prefer an init job, a CI/CD provisioning step, or declarative topic management that is owned by the platform team.

11.4.2 Init container

For Kubernetes deployments where the Kafka broker shares the cluster, add an init container to the generated `k8s.yml`:

```
initContainers:
```

```
  - name: topic-init
    image: bitnami/kafka:latest
    command: ['/bin/sh', '-c']
    args:
      - |
        /opt/bitnami/kafka/bin/kafka-topics.sh --create \
          --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
          --topic <name> --partitions 3 --replication-factor 1
```

11.4.3 Kafka operator (Strimzi)

In Kubernetes environments running the Strimzi operator, topics can be declared as `KafkaTopic` custom resources. A generated `kafka-topics.yml` CRD manifest can be applied alongside the Deployment:

```
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: <process>_<task>_input
  labels:
    strimzi.io/cluster: my-kafka
spec:
  partitions: 3
  replicas: 1
---
```

The scaffolder can generate Strimzi-oriented output when requested. Treat the generated manifest as a starting point: set replicas, partitions, retention and labels to match the target Kafka cluster policy before applying it.

11.5 Test environment deployment

A Docker Compose-based test environment is the fastest way to validate a pipeline end-to-end. The repository's `setup/docker-compose.yml` provides a single-node Kafka broker and Kafka UI.

Add a `docker-compose.test.yml` to the generated project:

```
version: "3"
services:
  pipeline:
    build: .
    environment:
      KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    depends_on:
      - kafka
  kafka:
    image: bitnami/kafka:latest
    environment:
```

```
      KAFKA_CFG_NODE_ID: 1
      KAFKA_CFG_PROCESS_ROLES: broker,controller
      KAFKA_CFG_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CFG_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CFG_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

Run with:

```
docker compose -f docker-compose.test.yml up --build
```

11.6 Production deployment

The generated `deploy.sh` supports three targets:

JAR `./deploy.sh` — builds the shaded JAR and runs locally. Suitable for development, VMs and bare-metal deployments when process supervision is provided externally.
Docker `DOCKER=true ./deploy.sh` — builds a Docker image tagged `<processId>:latest`.
Kubernetes `DOCKER=true K8S=true ./deploy.sh` — builds the image and applies the generated `k8s.yml`.

11.6.1 Docker image

The generated `Dockerfile` uses `eclipse-temurin:21-jre-alpine`, runs as a non-root user, and includes a process-based health check. In production, replace or supplement process checks with application-level readiness that verifies Kafka connectivity and worker readiness:

```
FROM eclipse-temurin:21-jre-alpine
RUN addgroup -S durga && adduser -S durga -G durga
USER durga
COPY target/*-all.jar /app/pipeline.jar
HEALTHCHECK --interval=30s ...
ENTRYPOINT ["java", "-Xms64m", "-Xmx256m", \
  "-Dkafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:-...}", \
  "-jar", "/app/pipeline.jar"]
```

11.6.2 Kubernetes

The generated `k8s.yml` provides:

- A `Deployment` with 2 replicas (configurable), baseline resource requests and limits,
- A `Service` exposing port 8080,
- Exec-based liveness and readiness probes.

11.7 Scaling

Workers scale horizontally via Kafka consumer groups. Adding pods increases concurrency only up to the partition count of the input topic. Scaling therefore requires topic partition planning, idempotent handlers, and a clear policy for external side effects. Three mechanisms are available, from manual to automated.

11.7.1 Manual scaling

```
kubectl scale deployment/<processId>-<taskId> --replicas=4
```

Suitable for development, staging, or controlled incident response. The operator watches consumer lag, processing latency, error rate and downstream service capacity before changing replicas.

11.7.2 KEDA autoscaling (volume-based)

When `-strimzi` or `-separate-workers` is active, the scaffolder generates `keda-scalers.yml` with one `ScaledObject` per worker. Each has dual triggers — Kafka consumer lag as the primary signal, and Prometheus latency as a quality-of-service safety net:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: invoice_receipt-review_invoice
spec:
  scaleTargetRef:
    name: invoice_receipt-review_invoice
  minReplicaCount: 1
  maxReplicaCount: 10
  triggers:
    - type: kafka
      metadata:
        bootstrapServers: kafka-broker:9092
        consumerGroup: invoice_receipt_review_invoice_worker
        topic: invoice_receipt_review_invoice_input
        lagThreshold: "100"
    - type: prometheus
      metadata:
        serverAddress: http://durga-monitoring:8081
        metricName: durga_activity_latency_ms
        threshold: "30000"
        query: durga_activity_latency_ms{pct="p95",
          process_id="invoice_receipt",
          activity_id="review_invoice"}
```

The Kafka trigger scales on message backlog (volume). The Prometheus trigger scales when processing latency degrades despite low lag — for example, when a worker is bottlenecked on an external service or database.

11.7.3 Custom metrics with Prometheus Adapter + HPA (Kubernetes-native)

An alternative for environments without KEDA: Prometheus scrapes `/api/metrics`, the Prometheus Adapter exposes custom metrics to the K8s metrics API, and an HPA scales the Deployment.

```
# ServiceMonitor (prometheus-operator)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: durga-monitoring
spec:
  selector:
    matchLabels: { app: durga-monitoring }
  endpoints:
    - path: /api/metrics
      interval: 15s
---
# Prometheus Adapter rule (in its ConfigMap)
- seriesQuery: 'durga_activity_latency_ms{pct="p95"}'
  name:
    matches: "durga_activity_latency_ms"
    as: "activity_latency_p95_ms"
---
# HPA
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: review-invoice-latency
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: invoice_receipt-review_invoice
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Pods
      pods:
        metric:
          name: activity_latency_p95_ms
        target:
          type: AverageValue
          averageValue: "30000"
```

11.7.4 Metrics reference

The monitoring app exposes at `/api/metrics`:

`durga_activity_latency_ms` Gauge. Labels: `pct` (p50, p95, p99, avg), `process_id`,

activity_id. Value is milliseconds.

durga_activity_samples Gauge. Labels: process_id, activity_id. Total completed samples.

durga_activity_sla_violations Counter. Labels: process_id, activity_id. Count exceeding the configured (global) SLA-latency threshold.

durga_alarm_active Gauge (1 per active alarm). Labels: process_id, activity_id, syndrome, severity. Every active alarm-state entry, including SLA alarms.

durga_alarm_fire_count Counter. Same labels as durga_alarm_active; number of firings folded into the active alarm.

durga_sla_last_observed Gauge. Labels: process_id, activity_id, syndrome. Latest observed SLA measurement — wall-clock latency in ms for SLA_LATENCY, or windowed completion count for SLA_THROUGHPUT.

durga_sla_limit Gauge. Same labels as durga_sla_last_observed; the agreed limit — maximum latency in ms, or minimum completions per window.

durga_streams_state Gauge. 1 if RUNNING, 0 otherwise.

11.8 Monitoring in production

The monitoring topology (ProcessMonitoringApp) is a separate deployment that consumes the canonical process-events topic and materializes queryable state. In production:

- Deploy the monitoring app as a companion service alongside the pipeline (same Kafka cluster, different consumer group).
- The Quarkus REST API and Svelte dashboard provide operational visibility — instance counts, latency summaries, stuck-instance detection, trend reporting.
- The monitoring topology uses global tables: every instance has a full local replica of the monitoring model. Any instance can answer all queries without cross-instance RPC.
- Protect the REST API and dashboard with network policy, ingress authentication, or an internal-only service boundary. Process instance views may include business keys, correlation ids and error messages.
- Retain the canonical lifecycle event topic for at least the longest monitoring recovery window you expect to support.

11.9 Configuration reference

Variable	Default	Description
KAFKA_BOOTSTRAP_SERVERS	localhost:9094	Kafka broker list
HTTP_PORT	8080	Quarkus HTTP port
PROFILE	dev	Quarkus configuration profile
START_KAFKA	false	Start Docker Compose Kafka before running
DOCKER	false	Build Docker image in deploy script
K8S	false	Apply Kubernetes manifests in deploy script

12 Supervision and Fault Tolerance

Durga runtime components communicate through Kafka topics, which act as durable mailboxes. This architecture enables an Erlang/OTP–like supervision model: workers can crash and restart independently while Kafka preserves unconsumed records according to topic durability and retention settings.

12.1 Architecture

- Every worker consumes from a dedicated Kafka topic (its inbox) and produces to downstream topics.
- Kafka persists messages for a configurable retention period. A crashed worker's unconsumed messages remain in its topic.
- Consumer group rebalancing shifts partitions to surviving instances when a worker pod exits.
- The scaffolder's `-separate-workers` flag generates one Kubernetes Deployment per BPMN task, giving each worker its own fault domain.

12.2 Per-worker isolation

When `-separate-workers` is enabled:

```
./durga durga-tools/src/test/resources/bpmn/invoice_receipt.bpmn \
--separate-workers
```

Each generated project includes:

- `RegisterInvoiceStandalone.java`, `ReviewInvoiceStandalone.java`, `NotifyRequesterStandalone.java` — one standalone worker per BPMN task, each with its own Kafka consumer/producer and poll loop.
- `InvoiceReceiptWorkerBootstrap.java` — entry point that reads the `WORKER_ID` environment variable and starts the matching worker.
- `Dockerfile.<task>` — per-worker Docker image building from the same JAR, with `WORKER_ID` pre-set.
- `k8s.<task>.yml` — per-worker Kubernetes Deployment with isolated resource limits, health probes, and `WORKER_ID` environment variable.

12.3 Supervision policies

Policy	Implementation
Crash recovery	Kafka consumer offset tracking. Worker restarts replay from last committed offset.
Fault isolation	Per-worker Deployments. One worker's OOM does not affect others.
Health signals	Exec-based liveness/readiness probes per pod (check process alive). Extendable to application-level health via `quarkus-smallrye-health`.

Scaling	`kubectl scale` per Deployment. Bounded by topic partition count.
Dead letter	Plugin executor routes failures to `%task%_dlq` topic. Standalone workers log errors and continue.
Backpressure	Kafka consumer group throttles ingestion. Consumer lag available via JMX metrics.

12.4 Recovery guarantees

Message durability Messages are not lost on worker crash. Kafka retains uncommitted records according to the topic's replication, acknowledgement and retention configuration.

At-most-once vs at-least-once Standalone workers commit after processing (`commitSync`). The poll loop catches and logs errors without crashing. Plugin-generated executors route failures to the DLQ. The architecture provides at-least-once delivery with idempotent output and external side-effect handling being the application owner's responsibility.

Process continuity If the starter crashes, the process never starts (one-shot job, retried by K8s). If a worker crashes, Kafka consumer group rebalancing reassigns its partitions. If the final orchestrator crashes, the process remains in-flight until a new orchestrator pod picks up.

12.5 Limitations

- Worker state is external (Kafka). In-memory state such as running aggregations or cancellation registrations is lost on crash unless that specific generated handler persists it.
- The `-separate-workers` model generates poll-loop workers rather than Quarkus CDI beans. These trade CDI features (dependency injection, reactive messaging) for isolated fault domains and smaller per-pod memory footprints.
- Cross-worker coordination (AND joins, subprocess completion) relies on the `ProcessStateStore`, which is itself a Kafka consumer. Its crash characteristics are the same as any other worker.
- Kafka durability does not imply business-level exactly-once semantics. Handlers that call databases, HTTP services or file systems must be idempotent or protected by application-level deduplication.

13 Data Assets

Durga treats BPMN data objects as logical pipeline assets, not as Kafka topics. Sequence flows still drive execution and Kafka still carries process events, worker input/output messages, BPMN messages and BPMN signals. Data objects describe the business artifacts that tasks read and write.

13.1 Model

A BPMN data object represents a named dataset or artifact, for example `RawCustomerData`, `NormalizedCustomerData`, or `CustomerGraphDelta`. A BPMN data store represents a physical source or target such as S3, PostgreSQL, Neo4j, a local file store, or an API.

Data input and output associations connect executable activities to those assets:

```
Transform Data
  input:  RawCustomerData
  output: NormalizedCustomerData

Enrich Data
  input:  FilteredCustomerData
  output: EnrichedCustomerData
  stores: S3Warehouse, Neo4jCustomerGraph, PostgresCustomerStore
```

The scaffolder collects these declarations into `summary.json` and the generated README. The declarations are metadata and contracts; they do not change routing by themselves.

13.2 Runtime handles

Large datasets should normally travel by reference rather than as Kafka message payload bytes. The generated runtime includes `DataHandle`:

```
DataHandle(
    name,
    uri,
    mediaType,
    schema,
    metadata)
```

A process event payload can carry a serialized data handle such as an S3 URI, media type, schema identifier, row count, checksum, or partition metadata. This keeps lifecycle events small while preserving enough information for monitoring and lineage.

Durga now includes experimental store plugins for this pattern. The `object-store-collector` writes the current payload to a local file-backed object store and forwards a `DataHandle`-style reference. The `object-store-extractor` resolves that reference and emits the referenced bytes to the next topic. Generated plugin and custom workers can also use `handleMode=manual` to pass handles through as plugin payloads, or `handleMode=materialize` to load the referenced bytes, run the plugin, and write the modified result back to the object store as a new handle. The current implementation is local-file backed; S3, GCS, Azure Blob, and other object-store adapters remain production hardening work.

13.3 Extension properties

Data objects and data stores may use Camunda extension properties for Durga metadata:

```
schema    = schemas/enriched-customer-data.schema.json
mediaType = application/json
kind      = s3
uri       = s3://warehouse/customer/enriched/
```

For data stores, `kind` identifies the adapter family. Current examples include `s3`, `postgresql`, and `neo4j`. If `kind` is omitted, later tooling may infer it from the URI scheme.

13.4 Lineage and metadata events

The implementation records data objects, data stores, and task associations in generated metadata. Generated plugin and custom workers also emit a Vannak-compatible `DataIndividualMetadataEvent` companion record to the `vannak-metadata-events` Kafka topic after successful execution. That event captures process/activity context, payload size and checksums, plugin identity/configuration, output fields, and `DataHandle`/format metadata when present.

The remaining direction is to bind BPMN data associations more tightly to those events, so task completions can report:

```
read:   [RawCustomerData]
wrote:  [EnrichedCustomerData]
stores: [S3Warehouse, Neo4jCustomerGraph]
```

This keeps the process engine focused on orchestration while giving pipeline operators the data-plane visibility needed for real workloads.

14 Kafka Connect Integration

Kafka Connect handles edge I/O. Durga owns the process and topic topology; Connect owns the movement between Kafka and external systems. The scaffolder's `-connect` flag generates connector skeletons for the pipeline's generic intake/output topics and, when BPMN data stores are present, for data-store-specific source and sink edges.

14.1 Architecture

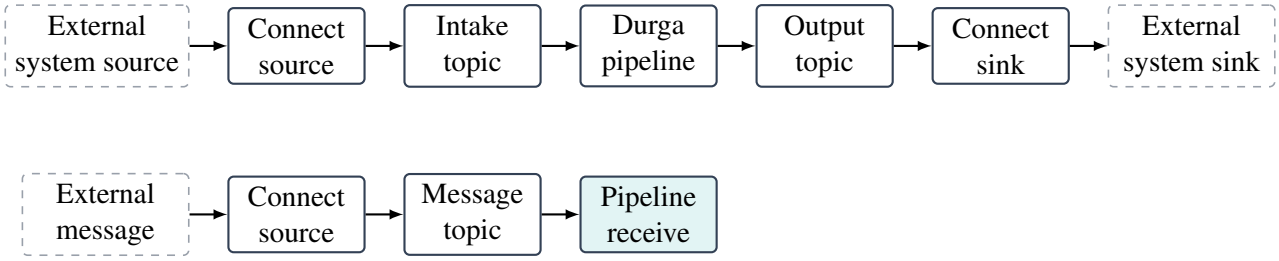

Figure 3: Kafka Connect integration: sources feed intake topics, sinks consume output topics

14.2 Enabling

```
./durga path/to/process.bpmn --connect
```

This adds a `connect/` directory to the generated project:

```
connect/connect-source.json
connect/connect-sink.json
connect/data-stores/*.json
connect.sh
```

14.3 Source connectors

`connect-source.json` contains connector configuration for:

- the process start topic (`%processId%_start`),
- message catch and receive task topics.

The user fills in the connector class and system-specific settings:

```
{
  "name": "invoice_receipt-source",
  "config": {
    "connector.class": "...",
    "tasks.max": "1",
    "invoice_receipt_start": {
```

```
      "kafka.topic": "invoice_receipt_start"
    }
  }
}
```

14.4 Sink connectors

`connect-sink.json` contains connector configuration for:

- terminal task outputs,
- the canonical `process-events` topic (for auditing or replication),
- message throw and send task topics.

14.5 Data-store connectors

When BPMN data stores are connected to tasks by data associations, Durga also generates one connector skeleton per data-store edge:

- a data store used as a task input source becomes a source connector skeleton that writes to the corresponding task input topic,
- a data store used as a task output target becomes a sink connector skeleton that reads from the corresponding task output topic.

For example, a task `enrich_data` that writes to `s3_warehouse`, `neo4j_customer_graph`, and `postgresql_customer_store` produces files such as:

```
connect/data-stores/data_pipeline_demo-s3-warehouse-sink.json
connect/data-stores/data_pipeline_demo-neo4j-customer-graph-sink.json
connect/data-stores/data_pipeline_demo-postgresql-customer-store-sink.json
```

These skeletons consume the existing Durga task output topic, for example `data_pipeline_demo_enrich_data_output`. The BPMN data object remains a logical asset; it is not converted into a Kafka topic.

14.6 Deploying

The generated `connect.sh` posts all connector JSON files under `connect/` and `connect/data-stores/` to the Connect REST API:

```
CONNECT_URL=http://kafka-connect:8083 ./connect.sh
```

Connector status can be checked with:

```
curl http://kafka-connect:8083/connectors/invoice_receipt-source/status
curl http://kafka-connect:8083/connectors/invoice_receipt-sink/status
curl http://kafka-connect:8083/connectors/data_pipeline_demo-s3-warehouse-sink/st
```

14.7 Common connector classes

Connector	Use case
`DebeziumSourceConnector`	CDC from PostgreSQL, MySQL, MongoDB
`JdbcSourceConnector`	Polling queries from relational databases
`S3SinkConnector`	Flush to S3-compatible object stores
`ElasticsearchSinkConnector`	Index to Elasticsearch
`HttpSinkConnector`	POST to REST APIs
`JdbcSinkConnector`	Write to relational databases
`Neo4jSinkConnector`	Write graph changes to Neo4j
`BigQuerySinkConnector`	Load into Google BigQuery

14.8 Limitations

- Connector configs are generated as stubs. The user must fill in connector-specific settings (connection URLs, credentials, table names, bucket names, converters and transforms).
- Connector class names for data stores are hints. The deployed Connect cluster must actually have the selected connector plugin installed.
- The scaffolder does not validate connector configs against a live Connect instance.
- Single Message Transform (SMT) chains are not generated. The user adds transforms in the connector config as needed.

15 Generated Code Extension Strategy

Generated code is a starting point. The BPMN model is the primary design source; generated code is scaffolding expected to be completed and adapted. Durga provides three strategies for extending generated code, ordered by preference.

15.1 Strategy 1: Use a registered plugin

For common data pipeline operations, use one of the 18 registered plugins. Annotate a `ServiceTask` with `camunda:plugin` and `camunda:pluginConfig` in the BPMN model. The scaffolder generates a complete, working worker that delegates to the plugin at runtime. No new code is required.

This covers: field remapping, filtering, type coercion, string templating, explicit field masking, regex extraction, JSON flatten/unflatten, UUID injection, timestamp normalization, schema validation, key-value enrichment, field-based routing, window counting, format detection, object-store collection/extraction through `DataHandle` references, and Kafka Connect source/sink configuration.

15.2 Strategy 2: Implement a custom activity

When no plugin covers the required logic, mark the BPMN activity with `camunda:plugin="custom"`. The scaffolder generates:

- A **contract interface** (e.g. `MyTransformContract extends Plugin`) that defines the API between generated and hand-written code.
- A **delegating worker** that injects the contract via CDI (`@Inject`), delegates each message through `PluginExecutionSupport.execute(...)`, and includes a null guard with dead-letter routing.

The developer writes an implementation class in a standard Java source file, compiles and tests it with the rest of the project. The generated contract interface and delegating worker are not modified by hand.

A build-time utility (`ModelEnricher`) scans compiled classes for implementations of generated contracts, matches them to BPMN activities, and writes `customImpl`, `customSource`, and `customHash` properties back into the local BPMN model copy. On regeneration, the enriched model carries the implementation references forward — the scaffolder regenerates the contract and worker but never touches the hand-written implementation class.

The BPMN model is copied into `src/main/resources/` in every generated project. It is version-controlled alongside the custom implementation and contract interface. Plugin-only workers can be `.gitignored` since they are fully regenerated from the model.

15.3 Strategy 3: Modify generated code directly

For one-off customizations that don't fit the plugin or custom activity models, modify the generated Java files directly. The scaffolder skips files that already exist in `src/main/java/` on regeneration, so modifications survive.

This approach is reserved for cases where strategies 1 and 2 are insufficient. Custom activity contracts separate generated code from hand-written logic, so regeneration does not overwrite custom work.

15.4 What is typically edited

Common edits in generated projects:

- custom activity implementations (implementing generated contracts),
- gateway condition expressions in the BPMN model (evaluated at runtime by the recursive descent parser — no code modification needed),
- plugin configuration strings such as `handleMode=manual`, `handleMode=materialize`, `store=...`, and plugin-specific `pluginConfig=...` values,
- payload handling and validation,
- integration calls inside custom implementations,
- operational configuration values in `application.yml`.

15.5 What should remain generated

Keep these close to the generated structure:

- topic and channel topology,
- lifecycle event emission shape,
- probe script set,
- subprocess and boundary wiring,
- monitoring-facing event semantics.

15.6 Regeneration strategy

When the pipeline needs structural changes (add/remove/reorder activities, change gateway logic, adjust plugin configuration), edit the BPMN model in Camunda Modeler and regenerate the project. For custom activities, start from the enriched model that already carries implementation metadata.

Plugin workers are regenerated from scratch. Contract interfaces are regenerated if the activity configuration changed. Hand-written implementation classes are never touched. If an implementation class was removed or renamed, the scaffolder emits a warning but does not delete the orphaned file.

16 Plugin Architecture

Durga supports a plugin-driven data pipeline model where BPMN `ServiceTask` elements can be annotated with `camunda:plugin` attributes that resolve to registered, governed data processing components.

16.1 Plugin registry

Plugins are defined by YAML descriptors under `plugins/` and indexed by `plugins/catalog.yml`. Each descriptor declares:

- `id` — unique plugin identifier (used in `camunda:plugin`),
- `name` / `version` — human-readable identifier,
- `category` — governed taxonomy (transform, validate, enrich, route, aggregate, inspect, store, connect),
- `status` — lifecycle state (experimental, stable, deprecated, retired),
- `implementation` — Java class implementing `org.gautelis.durga.plugins.Plugin`. For the Rust code-generation target (Section 18) the same plugin is implemented natively in the `durga-rust` crate under the same catalog identity and configuration schema; there are no adapters onto the Java implementation.
- `config` — typed configuration schema (field types, defaults, allowed values).

16.2 Governed categories

Each plugin category has an intended contract for topic naming, lifecycle events, and error channel conventions. The generated plugin executor now emits category-specific lifecycle events for the supported governed cases: `GATEWAY_TAKEN` for route plugins, `ACTIVITY_ESCALATED` for validate plugin failures, and, for aggregate plugins that keep a window open by returning no output, `PROCESS_COMPLETED` for the absorbed instance (it is folded into the window and reaches a terminal state rather than lingering as active). Other plugin categories retain the normal activity completion/failure lifecycle.

Category	Contract
transform	1:1 input→output, ACTIVITY_COMPLETED, dlq on failure
validate	Output + invalid branch, ACTIVITY_ESCALATED on invalid
enrich	Input→enriched output, may block on external lookup
route	Input→N downstream channels, GATEWAY_TAKEN lifecycle
aggregate	Windowed counting, ACTIVITY_ENTERED on window start
inspect	Payload inspection and metadata extraction without changing the data-plane artifact
store	Payload externalization and retrieval through `DataHandle` references
connect	Kafka Connect source/sink configuration generation

16.3 Plugin interface

All 16 processing plugins implement `org.gautelis.durga.plugins.Plugin`, which provides four `default` methods. The two connect catalog descriptors (`kafka-connect-source` and

kafka-connect-sink) are code-generation descriptors that reference the scaffolder, not runtime Plugin implementations.

```
byte[] execute(byte[] payload, String config) throws Exception
String execute(String payload, String config) throws Exception
PluginResult executeWithResult(byte[] payload, String config) throws
Exception
String idempotencyKey(byte[] payload, String config)
```

The generated worker always calls executeWithResult. Its default implementation delegates to the byte[] overload and wraps the output in a PluginResult carrying a content-based idempotency key and the default PAYLOAD disposition. Text / JSON plugins override the String execute variant (UTF-8 conversion is handled by the byte[] default); binary plugins override the byte[] variant directly. Plugins that need an explicit idempotency strategy, a non-PAYLOAD OutputDisposition (PASSTHROUGH for route/inspect control values, SIDE_EFFECT for stored-object handles), or a non-exceptional error strategy (ErrorStrategy.DLQ, SKIP, or FAIL) override executeWithResult to return a structured PluginResult. For example, the JSON Schema Validator honours an onInvalid=dlq|skip|fail directive by returning the corresponding error strategy.

The generated worker inspects the returned PluginResult.errorStrategy(): SKIP acknowledges and drops the message, DLQ routes it to the task's dead-letter topic, and FAIL fails the activity (also to the dead-letter topic, plus a PROCESS_FAILED lifecycle event).

Generated executor classes instantiate the plugin at field level and delegate through PluginExecutionSupport.execute(...) for each message. The config string comes from the BPMN camunda:pluginConfig property. Default execution is unchanged: the current ProcessEvent.payload JSON is passed to the plugin. Handle-aware execution can also be requested:

- handleMode=manual passes the DataHandle JSON to the plugin so the plugin can operate on the handle itself.
- handleMode=materialize reads the bytes referenced by dataHandle.uri, passes those bytes to the plugin, writes plugin output back to the configured object store, and forwards a new DataHandle.
- pluginConfig=... supplies the plugin's own configuration when the outer config string is used for handle-control options.

The returned bytes are decoded as UTF-8. JSON object results replace the ProcessEvent.payload. JSON scalar, JSON array, and non-JSON text results are wrapped as { "_value": ... }. A null result means the incoming payload is forwarded unchanged. Generated plugin and custom workers also emit a Vannak-compatible DataIndividualMetadataEvent to the vannak-metadata-events topic after successful execution.

16.4 Plugin catalog

The repository ships with 18 catalog descriptors organized by category (16 processing plugins implementing Plugin plus 2 connect code-generation descriptors). Stable plugins are fully supported; experimental plugins generate with a warning.

16.4.1 Transform plugins (9)

JSON Transform Dot-notation field remapping with colon-syntax renaming and literal injection.
Field Filter Whitelist/blacklist field filtering with optional flatten.
Type Coercion Coerces field values between string, int, long, double, decimal, and boolean.
String Template Renders a template string with `${field}` substitution. Dot-notation access for nested fields.
Mask Masks explicitly configured text fields with configurable mask character and boundary character preservation.
Regex Extract Extracts named capture groups from a source field into the payload. Supports multiple matches.
JSON Flatten Flattens nested JSON to dot-notation keys, or unflattens dot-notation back to nested objects. Configurable separator and max depth.
UUID Inject Injects UUIDs (v7, v4 or v1) into specified payload fields.
Timestamp Normalize Converts between epoch_s, epoch_ms, ISO 8601, RFC 3339, and custom DateTimeFormatter patterns. Configurable timezone.

16.4.2 Validate plugins (1)

JSON Schema Validator Validates payloads against a JSON Schema subset. Routes invalid messages to DLQ, skipped, or fails. Status: experimental.

16.4.3 Enrich plugins (1)

KV Enricher Looks up a key field value in an inline data map and shallow-merges enrichment data. Status: experimental.

16.4.4 Route plugins (1)

Field Router Routes messages to named output channels based on a field value. Status: experimental.

16.4.5 Aggregate plugins (1)

Window Counter Counts messages within a tumbling time window and emits a summary record on window close. Optional group-by field. Status: experimental.

16.4.6 Inspect plugins (1)

Format Detector Detects MIME type, textual/binary shape, JSON object/array/scalar form, byte count, and checksums. Status: stable.

16.4.7 Store plugins (2)

Object Store Collector Writes the current payload to a local file-backed object store and emits a `DataHandle`-style JSON reference. Status: experimental.
Object Store Extractor Resolves a `DataHandle`-style object reference and emits the referenced bytes. Status: experimental.

16.4.8 Connect plugins (2)

Kafka Connect Source Generates Kafka Connect source connector configuration for pipeline intake topics.
Kafka Connect Sink Generates Kafka Connect sink connector configuration for pipeline egress.

16.5 BPMN binding

Plugin-annotated tasks use Camunda extension properties on `ServiceTask` elements:

```
<bpmn:serviceTask id="transform" name="Transform Data">
  <bpmn:extensionElements>
    <camunda:properties>
      <camunda:property name="plugin" value="json-transform" />
      <camunda:property name="pluginConfig" value="name, email" />
    </camunda:properties>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

Without a `camunda:plugin` attribute, `ServiceTask` elements fall back to the existing generic worker generation.

16.6 Generated executor

The scaffolder resolves the plugin descriptor, validates it against the registry, and generates a concrete executor class using the `pluginExecutorClass` template. The executor:

- imports the plugin implementation class and the `Plugin` interface,
- imports `PluginExecutionSupport` for handle-aware execution,
- instantiates the plugin as a private field,
- delegates each incoming message to `PluginExecutionSupport.execute(...)`,
- emits `ACTIVITY_COMPLETED` on success,
- emits a companion `DataIndividualMetadataEvent` to `vannak-metadata-events` on success,
- routes failures to a dead-letter topic (`%task%_dlq`) with structured error records,
- respects `ScopeCancellationRegistry` for interrupting boundary support.

16.7 Custom activities

When no registered plugin covers a transformation, the `camunda:plugin="custom"` marker triggers a different code generation path designed for hand-written implementations:

```
<bpmn:serviceTask id="custom_transform" name="Custom Transform">
  <bpmn:extensionElements>
    <camunda:properties>
      <camunda:property name="plugin" value="custom" />
      <camunda:property name="pluginConfig"
                        value="org.example.CustomTransformContract" />
    </camunda:properties>
  </bpmn:extensionElements>
</bpmn:serviceTask>
```

Instead of generating a self-contained worker, the scaffolder produces:

1. A **contract interface** (`CustomTransformContract`) extending `Plugin` — this defines the API between generated and hand-written code.
2. A **delegating worker** (`CustomTransformWorkerService`) that injects the contract via CDI (`@Inject`) and delegates each message through `PluginExecutionSupport.execute(...)`. The worker includes a null guard that routes to the dead-letter queue if no implementation is present.

The developer writes an implementation class (`CustomTransformImpl implements CustomTransformContract`) in a standard Java source file. It is compiled and tested with the rest of the project. The generated contract and worker are not modified by hand.

16.7.1 Round-trip model enrichment

A build-time utility (`ModelEnricher`) scans compiled classes for implementations of generated contracts, matches them to BPMN activities by contract name, and writes `customImpl`, `customSource`, and `customHash` Camunda extension properties back into the local copy of the BPMN model. Detection uses a `URLClassLoader` and reflection (`isAssignableFrom`); hashing uses the source file (`.java`) rather than the bytecode (`.class`), so the hash is stable across JDK versions and compiler flags. The source directory is derived from the classes directory by Maven convention (`target/classes` → `src/main/java`) or passed explicitly.

The enriched model becomes the source of truth for future regenerations. When the pipeline changes, regeneration starts from the enriched model — it already knows which activities have custom implementations and which Java class backs each one. The custom implementation files are never touched by the scaffolder.

16.8 Current limitations

- Plugin descriptors are read from the filesystem or classpath at scaffold time — there is no runtime plugin discovery or hot-reload.
- Config validation at scaffold time is not yet implemented — invalid configs are caught at runtime by the generated executor.

- The plugin catalog must be maintained manually; there is no registry publishing or version-resolution tooling.

17 Generated Runtime Classes

Each generated project contains Java classes under the configured base package (`-package`, default `org.example.ge`
In the examples below this package is written as `<pkg>`. Operational classes (workers, gateways, orchestrator) are generated under: `src/main/java/<pkg>/`. Probes and exercisers (starter, scenario runner, publishers, watchers) are generated under: `src/main/java/<pkg>/probes/`. Shared runtime contract classes (`ProcessEvent`, `DataHandle`, plugin runtime, etc.) are generated under `src/main/java/org/gautelis/durga/`. These classes map BPMN constructs onto Kafka consumers, producers and lifecycle events rather than hiding behind an opaque engine.

17.1 Class families

- **Starter classes** publish the initial lifecycle event and first task input.
- **Task services** handle BPMN tasks. Service-task handlers auto-complete after their work step; user and manual task handlers enter a waiting state and await explicit completion, failure or escalation input.
- **Plugin executors** instantiate a registered plugin at field level and delegate each message through `PluginExecutionSupport.execute(...)`. No per-plugin code paths in the generator.
- **Custom contract interfaces** define the API for hand-written implementations of custom activities. They extend the `Plugin` interface.
- **Custom delegating workers** inject a contract interface via CDI and delegate each message through `PluginExecutionSupport.execute(...)`. Includes a null guard with dead-letter routing.
- **Data and metadata contracts** include `DataHandle`, `DataIndividualMetadataEvent`, `VannakMetadata`, and `PluginExecutionSupport` so generated workers can externalize large payloads and emit item-level metadata events.
- **Gateway services** handle XOR, AND and OR routing decisions, including fan-out and join behavior. XOR and OR conditions are evaluated at runtime by a recursive descent expression parser supporting `==`, `!=`, `>`, `<`, `>=`, `<=`, `&&`, `||`, `!`, parentheses, and nested field access.
- **Event handlers** implement timer, message and signal behavior for intermediate events and boundary-event reactions.
- **Call-activity helpers** model request and reply behavior for call activities.
- **Subprocess scope services** emit subprocess entry and completion lifecycle events and coordinate subprocess-local routing.
- **Process orchestrator** observes terminal flows and emits process completion.
- **Probe helpers** provide generated publishers and watchers for command-line exploration.
- **Model registration** publishes the process's BPMN 2.0 model to the `process-models` Kafka topic on startup via a `@PostConstruct` CDI bean. This enables self-registration with the monitoring server and makes the process diagram available in the dashboard without pre-configuration.

17.2 Task services

Generated task services consume a task input topic, emit `ACTIVITY_ENTERED`, perform the generated action or enter a wait state, then emit `ACTIVITY_COMPLETED`, `FAILED`, `ESCALATED` or `CANCELLED` before forwarding to the next topic or boundary path.

17.3 Plugin executors and custom workers

Plugin executors and custom delegating workers share the same structure: a CDI bean with `@Incoming` for the task input topic and `@Channel` emitters for output, process-events, and dead-letter topics. The difference is in how the transformation logic is obtained:

- **Plugin executor**: instantiates the concrete plugin class as a field (`private final Plugin plugin = new JsonTransform();`) and delegates through `PluginExecutionSupport.execute(plugin, payload, config)`.
- **Custom delegating worker**: injects the contract interface via CDI (`@Inject MyContract implementation;`) and delegates through `PluginExecutionSupport.execute(implementation, payload, config)`. The implementation is provided by a developer-written class annotated with `@ApplicationScoped`.

Both patterns include scope cancellation checks, DLQ routing on failure, and canonical `process-events` emission. They also emit a Vannak-compatible `DataIndividualMetadataEvent` to the `vannak-metadata-events` topic after successful execution. For both patterns, returned bytes are interpreted as UTF-8. If the result parses as a JSON object, that object becomes the next `ProcessEvent.payload`. If the result is a JSON scalar, a JSON array, or non-JSON text, the generated worker wraps it as `{ "_value": ... }`. A `null` result keeps the incoming payload unchanged.

`PluginExecutionSupport` keeps default plugin behavior backward compatible while adding handle-aware modes. `handleMode=manual` passes the `DataHandle` JSON to the plugin. `handleMode=materialize` reads the referenced bytes from the configured object store, runs the plugin on those bytes, writes the output back to the store, and forwards a new `DataHandle`.

17.4 Routing and event classes

Gateways turn BPMN branching into explicit topic routing. Timer, message and signal handlers make delays and external triggers concrete. Boundary handlers observe task or scope state and branch into exceptional or reminder flows when trigger conditions are met.

17.5 Scope classes

Subprocess scope services emit subprocess-level entered and completed events, mediate flow into and out of the subprocess graph, propagate failure, escalation and cancellation at scope level, and support nested subprocess structure. Event subprocess handlers model subprocess-internal event-driven branches separately from the main task path.

18 Code-Generation Targets

The scaffolder is target-parameterised: from a single BPMN model it can render a generated project in more than one implementation language. The mature target is Java (Quarkus / SmallRye Reactive Messaging); a Rust target is in progress. Both targets emit workers that speak the same Kafka wire protocol, so a single process can, in principle, be realised as a mix of Java and Rust workers, and the same Java monitor observes either.

18.1 Template organisation

Template groups are separated by target under `durga-tools/src/main/resources/`:

- `templates-java/` — the Java target (`scaffold.stg` and companions). This is the default and the only fully implemented target.
- `templates-rust/` — the Rust target (in progress).

The attribute model passed from the generator (process id, task id, plugin reference, plugin configuration, channels, lineage) is identical across targets; only the rendered syntax differs. Loading the template group is the single switch point in `BpmnScaffolder`.

18.2 Shared source of truth

To keep the targets in lock-step, the following remain single sources of truth consumed by both:

- **Plugin catalog** — the YAML descriptors under `plugins/` define plugin identity, category, and the typed configuration schema. A model scaffolds identically for either target; the same plugin configuration string (e.g. `id:order_id, customer.email:customer_email`) is honoured by the Java and Rust plugin implementations alike.
- **Lifecycle wire format** — the `ProcessEvent` JSON (camelCase fields, SCREAMING_SNAKE event types and statuses) is byte-compatible across targets, so the monitoring topology, alarm model, and dashboard treat Rust-generated tasks exactly as Java tasks.

18.3 The Rust plugin runtime (`durga-rust`)

The `durga-rust/` Cargo crate is the Rust counterpart to the Java `durga-runtime` plugin layer. Generated Rust workers depend on it directly — there are no adapters onto the Java plugins; each plugin is a native Rust implementation.

The `Plugin` trait mirrors the Java `Plugin` interface, including its dispatch defaults:

- `execute_bytes(payload, config)` — binary entry point; the default UTF-8 decodes and delegates to `execute_str`, then re-encodes. Binary plugins override this directly.
- `execute_str(payload, config)` — text / JSON entry point; text plugins override this. The default is unsupported, matching the Java default.
- `execute_with_result(payload, config)` — the structured entry point the generated worker calls; the default wraps `execute_bytes` with a content-based idempotency key and the `PAYLOAD` disposition.

- `idempotency_key(payload, config)` — SHA-256 of payload then config, byte-identical to the Java default.

`PluginResult` carries the same `OutputDisposition` (`PAYLOAD`, `PASSTHROUGH`, `SIDE_EFFECT`) and `ErrorStrategy` (`FAIL`, `SKIP`, `DLQ`) semantics as the Java record, so routing and inspection plugins declare `PASSTHROUGH` and the generated worker preserves the payload identically on either target.

18.4 Validation shadow workers

Validation mode (Section 9) is emitted on both targets. With `-validation`, each plugin task gains a shadow worker beside the production worker: the Java target generates a *Name*`ValidationWorker` bean; the Rust target generates a `%taskId%_validation` binary that calls `run_validation_worker`. Both consume the production input topic through a dedicated consumer group, suppress side effects, and emit a `ValidationCandidateOutput` to the shared `validation-candidate-outputs` topic. That record is byte-compatible across targets, so the same Java monitor comparator pairs Java- and Rust-produced candidates against production without distinction. The Rust crate provides the matching `ValidationCandidateOutput` type and the pure `plan_validation_output` decision function, mirroring the Java `ValidationCandidateOutput` and the generated shadow-worker logic.

18.5 Status

Implemented so far: the crate contracts (`Plugin`, `PluginResult`, `ProcessEvent`, `ValidationCandidateOutput`, dot-path field helpers), the pipeline plugin ports, the pure worker decision logic (`plan_worker_output`, `plan_validation_output`), the Rust worker / gateway / validation and project (`Cargo.toml`, `lib.rs`, `build.rs`) templates, and the `--target rust` switch in the CLI. Rust workers use a mature Kafka client (`rdkafka`) and an async runtime (`tokio`) for the same consume–transform–emit loop the plain-Kafka Java worker performs. Rust remains the less-mature target: it currently covers plugin tasks, custom-task stubs (with contract modules and `build.rs` entries), exclusive gateways, and validation-mode shadow workers in a start/tasks/gateways/end topology, and reports and skips unsupported constructs.

19 Naming Conventions

Durga relies heavily on predictable names because the generated topology is intentionally visible. Naming is therefore part of the system design, not only a cosmetic detail.

19.1 Common naming patterns

The generator generally follows these patterns:

- process identifiers come from BPMN process ids,
- task topics use process and task identifiers,
- generated classes use process and BPMN element names converted to Java class names,
- helper scripts use short imperative file names such as `complete-task.sh`,
- the canonical lifecycle-event topic is per process by default, `process-events-%processId%` (override with `-event-topic`); the monitor consumes the whole `process-events-*` family,
- shared monitoring output topics use fixed names such as:

  ```
  process-state
  process-state-counts
  ```

- validation mode uses a dedicated consumer group `%processId%_%taskId%_validation` on the production input topic and the shared topics `validation-candidate-outputs` and `validation-results` (Section 9).

19.2 Why this matters

Consistent names make it easier to:

- inspect topic traffic in Kafka tools,
- map BPMN elements to generated Java handlers,
- reason about subprocess and boundary scope behavior,
- debug routing and completion behavior quickly.

19.3 Normalization and transliteration

Process ids and element names become both Kafka topic segments and generated Java identifiers, so they are normalized to a stable ASCII slug (`[a-z0-9_]`) by the shared `org.gautelis.durga.NameNormalizer`. The monitor uses the same class when it re-derives ids from a registered model, so lifecycle events and alarm/SLA configurations always agree.

Normalization is: transliterate to ASCII, lowercase, collapse any run of other characters to a single underscore, and trim leading/trailing underscores. Transliteration is not lossy dropping — accented Latin letters are mapped to a sensible ASCII equivalent. Swedish `å`/`ä`/`Å`/`Ä` become `a`/`A` and `ö`/`Ö` become `o`/`O` (via Unicode NFD decomposition), and a small table covers letters that do not decompose (`ø`→`o`, `æ`→`ae`, `ß`→`ss`, `þ`→`th`, `ð`→`d`, `ł`→`l`, ...). For example, a process named `Återköp` yields id `aterkop`, topic `process-events-aterkop`, and starter class `AterkopStarter`; a task `Registrera ärende` yields class `RegistreraArendeWorkerService`.

19.4 Scaffold-time validation

Because a broken or ambiguous id would produce invalid Java or colliding topics, the scaffolder validates all names before generating and **aborts** (with a message on standard error) when any of these hold:

- an id or name has no usable ASCII letters/digits after transliteration (e.g. an all-CJK id) and cannot yield an identifier,
- a normalized name starts with a digit (not a valid Java identifier),
- a normalized name is a Java reserved word,
- two distinct names normalize to the same slug (they would share generated classes and topics).

A non-fatal warning is emitted whenever a name was transliterated, so the drift from the source model is visible.

19.5 Practical guidance

For the cleanest generated output, BPMN ids should be stable, readable and explicit. If BPMN models use unclear or heavily tool-generated ids, the generated Kafka and Java output becomes harder to read as well.

If the BPMN tool changes element identifiers between edits, pin the process id with `-process-id <id>` to keep topic names, consumer groups, and class names stable across regenerations. The monitor registers each model under this effective id, so alarm and SLA configurations extracted from the model are scoped to the same id the running workers use.

20 Generated Project Lifecycle

Generated projects are used iteratively, not as one-off code dumps:

1. model a BPMN process,
2. scaffold a project from that model,
3. inspect generated topics, classes and helper scripts,
4. run demo flows or manual probes,
5. observe the lifecycle stream and monitoring views,
6. revise the BPMN model and regenerate.

20.1 Two modes of use

- as a learning scaffold, where the value is seeing how BPMN maps onto Kafka,
- as a starting point for a real implementation, where the generated code becomes a base that is extended, completed and hardened.

The BPMN model remains the primary design artifact. Generated output is readable, runnable and observable by design.

21 Demo Flows And Helper Scripts

21.1 Purpose

Generated scripts provide runnable examples of the Kafka topology, reduce the amount of manual producer and consumer work needed during exploration, and make BPMN semantics visible through actions such as completion, failure, escalation and observation.

21.2 Common generated scripts

The exact set depends on the BPMN model. Common script categories include:

- `demo-scenario.sh` for whole-process happy, stuck or failed runs,
- `send-task-input.sh` for manual injection into one task input topic,
- `complete-task.sh`, `fail-task.sh` and `escalate-task.sh` for human or exceptional task outcomes,
- `complete-call-activity.sh` for returning from a generated call activity,
- `send-message-event.sh` and `send-signal-event.sh` for external event stimulation,
- `watch-process-events.sh` and `watch-task-output.sh` for runtime observation.

21.3 Recommended learning loop

1. generate a project from one small BPMN sample,
2. create the Kafka topics and start the generated runtime,
3. run `watch-process-events.sh`,
4. execute `demo-scenario.sh happy`,
5. repeat with a failure, escalation or timeout-oriented script,
6. inspect the monitoring endpoints or dashboard after each run.

This loop exposes the difference between nominal and exceptional flow with very little setup. The execution topics carry work, but the canonical `process-events` stream carries the lifecycle truth.

22 Configuration And Operations

22.1 Scaffolder invocation

The repository includes a convenience script at `durga`:

```
./durga path/to/process.bpmn [flags...]
```

It auto-builds the JAR if missing and passes all arguments through to the scaffolder. Equivalent to:

```
mvn -q package -DskipTests
java -jar durga-tools/target/durga-tools-0.1.0-beta.1.jar path/to/process.bpmn [f
```

Available flags: `-dry-run`, `-out <dir>`, `-transactions`, `-separate-workers`, `-strimzi`, `-connect`, `-validation`, `-target java|rust`, `-event-topic <topic>`, `-process-id <id>`, `-package <pkg>`, `-retention <hours>`.

`-validation` additionally generates a validation-mode shadow worker for each plugin task (Java or Rust), so a candidate implementation can be run against real input on a dedicated consumer group with side effects suppressed and its output compared against production. It also registers the `validation-candidate-outputs` and `validation-results` topics. See Section 9. The default output (without the flag) is unchanged.

`-retention <hours>` sets Kafka topic retention in hours (defaults to 168 h/7 days with a warning). Use `-1` for infinite retention. The canonical event topic (configured via `-event-topic`) always has infinite retention; the `%process%_state` topic always uses compaction. Wired into both `topics.sh` and Strimzi `topics.yml`. Accepts `h`, `d`, `w`, `m` suffixes for hours, days, weeks, or 30-day months.

For production, do not accept generated retention defaults without review. Retention is part of the process contract: it controls replay windows, diagnostic history, storage cost and monitoring recovery. Infinite retention requires an explicit storage and compaction strategy.

`-package <pkg>` sets the Java package for generated code (defaults to `org.gautelis.durga.generated` with a warning). Operational classes are placed directly in the package; probe and exerciser classes in `<pkg>.probes`.

`-event-topic <topic>` overrides the physical Kafka topic name for canonical lifecycle events. The default is `process-events-<processId>`, so each pipeline writes to its own topic by default. Use this flag to direct multiple pipelines to a shared topic (e.g. `-event-topic process-events`) or to use a custom naming scheme. The logical channel names `process-events` and `process-events-monitor` in the generated code stay unchanged; only the physical topic mapping in `application.yml` is affected.

`-process-id <id>` overrides the process identifier read from the BPMN file. This is the name used in topic names, consumer group IDs, generated class names, and the `durga-<id>` consumer group prefix for the `ProcessStateStore`. Pinning the process ID keeps Kafka group coordination stable if the BPMN tool regenerates with different element identifiers.

22.2 Local runtime baseline

The normal local environment consists of:

- the bundled Kafka setup under `setup/`,
- generated topic configuration in the scaffolded project,
- the optional monitoring app and dashboard.

The repository is intentionally optimized for a local single-broker loop so the generated processes and monitoring projections can be explored quickly. This local baseline is not a production topology; production deployments need broker replication, authenticated clients, managed secrets, resource limits, topic ownership and backup/replay policy.

22.3 Important configuration locations

The main configuration locations are:

- repository-level runtime defaults in `src/main/resources/application.yml`,
- generated-project channel configuration in the scaffolded `application.yml`,
- local Kafka infrastructure in `setup/docker-compose.yml`,
- generated topic setup scripts in scaffolded output directories.

22.4 Operational expectations

In normal local startup:

- Kafka UI may log temporary connection failures before the broker is ready,
- generated runtimes rely on explicit topics and channels rather than implicit discovery,
- the monitoring app may report a transitional Streams state before its stores become queryable.

22.5 Where state lives

Durga uses several layers of state:

- transport history on execution topics,
- canonical lifecycle history on `process-events`,
- materialized latest state and counts in Kafka Streams stores and corresponding topics,
- local waiting and cancellation bookkeeping inside some generated handlers.

Operationally, the first three layers are Kafka-backed and can be recovered or replayed within their configured retention windows. The last layer is process memory and should be treated as transient unless the generated handler explicitly persists its coordination state.

22.6 Recommended local development loop

For practical local work:

1. start Kafka,
2. build the repository or generated project,

3. create topics,
4. run the generated process or a demo publisher,
5. start the monitoring app if query visibility is needed,
6. inspect lifecycle events, counts and latency.

23 Testing And Verification

23.1 Generator verification

Three layers:

- dry-run tests that treat scaffold summaries and previews as a contract,
- generation tests that write projects to temp directories and verify output (file existence, plugin imports, condition expressions, contract interfaces),
- a compilation integration test that generates a full project from `invoice_receipt.bpmn` and compiles all `.java` files with `javax.tools.JavaCompiler`, verifying the generated code is syntactically and semantically correct.

The verification set spans 26 BPMN fixtures covering:

- message and signal events,
- timers and boundaries,
- subprocesses and nested subprocesses,
- call activities,
- event subprocesses (message, signal, timer, error, escalation),
- data pipelines (plugin-annotated tasks, custom activities),
- gateway condition expressions.

23.2 Plugin verification

Each of the 16 Java plugin implementations has dedicated unit tests covering normal operation, edge cases, and error handling. The plugin registry loading and descriptor validation are exercised by the scaffolder tests. Dedicated tests also cover format detection, object-store collection and extraction, handle-aware plugin execution, and Vannak metadata event creation.

23.3 Data pipeline verification

Four dedicated tests verify:

- the plugin-annotated pipeline (`data_pipeline_demo.bpmn`) generates correct plugin executor classes and data-asset metadata,
- the order events pipeline (`order_events_pipeline.bpmn`) generates workers for 8 plugin types and an XOR gateway with runtime-evaluated conditions,
- the log processing pipeline (`log_processing_pipeline.bpmn`) generates workers for regex extraction, templating, flattening, and validation plugins,
- the custom activity demo (`custom_activity_demo.bpmn`) generates contract interfaces and delegating workers.

23.4 Connect verification

Connect tests verify that `-connect` generates generic source and sink connector configs with correct topic lists for both dry-run and full generation modes. A dedicated data-store test verifies that BPMN data stores generate per-store connector skeletons under `connect/data-stores/`, including S3, Neo4j and PostgreSQL sink examples.

23.5 Model enricher verification

Three tests verify the `ModelEnricher`: enrichment of custom activity metadata, no-op when no implementation is found, and property update on re-enrichment. Tests compile real Java source files during execution and verify that `customSource` references the `.java` path (not the `.class` file) and that hashes are computed from source files.

23.6 Core runtime contract verification

- `ProcessEvent` serialization round-trips and compact-constructor inference,
- `ProcessState` serialization round-trips,
- `ScopeCancellationRegistry` cancellation tracking, merging, clearing and null-guard behavior,
- `ProcessStateView` projection tests covering cancellation, escalation, process completion, gateway events, concurrent activities, and state-key generation.

23.7 Monitoring verification

- activity latency statistics (min, max, avg, p50, p95, p99, SLA violations),
- demo publishers and scenario runners that drive the monitoring topology without a full generated process runtime,
- fault detection unit tests (14 scenarios) covering the three syndromes (HARD_ERROR, COUNTED, SLIDING_WINDOW), config scoping, event type filtering, alarm suppression, and re-arm behaviour (see Section 6.5).
- alarm state unit tests covering state-key generation, repeated firing fold-up, severity preservation, late older events, and topology construction.

23.8 CI pipeline

The non-Docker verification command `mvn test -Dtest='!*IntegrationTest'` currently runs 355 tests (durga-runtime 215, durga-tools 63, durga-monitor 77). Docker-backed integration tests are suffixed `IntegrationTest` and are run separately for release gates.

23.9 Integration tests

Seven integration test suites use Testcontainers to spin up real Kafka brokers and verify runtime behaviour end to end. Each suite is documented in detail in Appendix 1.

ProcessEventKafkaIntegrationTest Serialization and JSON round-trip integrity over a live Kafka broker. Verifies that `ProcessEvent` payloads, status codes, error info, and compact-constructor inference survive wire transport intact.

ProcessStateStoreIntegrationTest Compacted-topic state persistence via `ProcessStateStore`. Exercises save/load, latest-wins semantics, multiple keys, not-found, and timeout handling against a real compacted Kafka topic.

MonitoringTopologyIntegrationTest The monitoring topology end to end: state materialization, state counts, activity latency summaries, stuck instance detection, and unkeyed event handling.

ChaosIntegrationTest Topology resilience under restart: verifies state recovery after crash, absence of duplicate state entries, and three consecutive restarts without corruption.

E2EPipelineIntegrationTest Full data pipeline end-to-end test. Verifies the scaffolder generates category-specific lifecycle events (`GATEWAY_TAKEN` for route plugins, `ACTIVITY_ESCALATED` for validate plugins, `ACTIVITY_ENTERED` for aggregate plugins), and that the monitoring topology correctly materializes state and latency from these event types in a mixed-type lifecycle.

ValidationTopologyIntegrationTest The validation (shadow-run) topology end to end: joins candidate-task outputs against production outputs over a live Kafka broker and verifies the comparison outcomes (`EQUAL`, `DIFF`, `PRIOR_MISSING`, `CANDIDATE_ERROR`) are materialized into the validation results store.

SlaMonitoringIntegrationTest The performance-SLA path end to end: runs the fault-detection and alarm-state topologies over a live broker and verifies that a slow task breaches its `SLA_LATENCY` ceiling and a starved pipeline breaches its `SLA_THROUGHPUT` floor, with both alarms materialized into the alarm-state read model that `/api/metrics` exposes.

23.10 Automation

The script `scripts/run-e2e-test.sh` automates the full test cycle:

```
./scripts/run-e2e-test.sh test       # Integration tests (Testcontainers)
./scripts/run-e2e-test.sh feed       # Infra + monitor + data feed
./scripts/run-e2e-test.sh test-run   # Tests + monitor + continuous feed
./scripts/run-e2e-test.sh stop       # Tear down
```

The `feed` and `test-run` commands build and start the monitoring server (port 8081), launch a Redpanda broker, PostgreSQL, and Kafka Connect via Docker Compose, register the E2E BPMN model, then start a lifecycle feed for live dashboard observation.

23.11 Not fully covered

- load and concurrency behavior,
- multi-instance monitoring scenarios.

24 Troubleshooting

Most operational failures are caused by startup timing, missing topics, incomplete task logic, Kafka connectivity, or confusion between execution state and monitoring state. Start with the runtime signals before assuming a generator defect.

24.1 Kafka UI logs connection failures at startup

This is usually normal in the local setup. Kafka UI often starts before the broker is fully ready and logs transient connection failures. If the broker then comes up and the errors stop, this is not itself evidence of a Durga runtime failure.

24.2 Monitoring app says the state store is not queryable yet

This normally means Kafka Streams has started but the state stores are not yet ready for queries. The correct response is usually to wait briefly and retry rather than to assume the topology is broken immediately.

In production, repeated or long-lived readiness failures should be treated as an operational incident. Check broker reachability, lifecycle-topic consumer lag, Streams application id stability, changelog topic health and whether a deployment started with a fresh state directory or missing permissions.

24.3 A generated process compiles but does not advance

The most common causes are:

- topics were not created,
- the relevant generated application is not running,
- an XOR branch condition still contains placeholder logic,
- a user or manual task entered a waiting state and needs explicit completion,
- the process instance was cancelled by a boundary or interrupting event subprocess,
- the worker processed a record but failed before committing its offset, causing a replay that requires idempotent handler logic.

24.4 Boundary behavior does not appear to trigger

Check these points first:

- the triggering lifecycle event is actually emitted,
- the boundary is interrupting or non-interrupting as intended,
- the attached task or subprocess reached the expected state,
- failure or escalation was emitted through the generated helper path rather than by bypassing the lifecycle model.

24.5 An external completion arrives after cancellation

The generated runtime now suppresses much of this post-cancel progress, but late events can still be confusing during debugging. In such cases, the canonical `process-events` stream is the best source of truth: check whether cancellation happened before the late completion arrived.

24.6 Monitoring counts do not match raw topic intuition

This is usually a model misunderstanding rather than a bug. Execution topics contain history and routing traffic. The monitoring counts are derived from the latest projected instance state, which is the intended read model for "how many instances are in state X right now".

24.7 Dashboard trend or latency data looks incomplete

The dashboard reads materialized monitoring stores, not raw execution topics. Missing trend or latency rows usually means no supported lifecycle event has reached the monitoring topology yet, the Streams stores are still restoring, or old monitoring topics contain records produced by an older projection version. For persistent inconsistencies, compare `process-events`, `process-trends`, `process-latency`, and the monitoring application logs.

24.8 The generated output looks harder to understand than expected

This often comes from BPMN naming. Unclear BPMN ids produce unclear Java classes, topics and helper artifacts. Cleaning up the BPMN model identifiers usually improves the generated output more than post-processing the generated files.

25 Known Design Tensions

25.1 Explicitness versus compactness

The generated runtime is larger than a hidden workflow runtime would be. This is accepted because the BPMN-to-Kafka mapping is visible, making the system easier to inspect and debug.

25.2 BPMN fidelity versus pragmatic Kafka mapping

BPMN semantics are projected through explicit Kafka handlers and topics rather than a full engine. Some semantics are intentionally pragmatic rather than exhaustive.

25.3 Regeneration versus local customization

Generated projects invite local edits, but the BPMN model remains the primary artifact. When the model changes substantially, regeneration with re-application of focused logic changes is preferred over manual rewiring of the generated topology.

25.4 Exploration versus production hardening

The system is strong enough for architectural exploration, but not every part is optimized for hardened production operation. The current focus remains clarity, runnable demos, and generator correctness.

26 BPMN Sample Catalog

The repository contains a growing set of BPMN models under `durga-tools/src/test/resources/bpmn`. They serve two purposes:

- they are regression inputs for the scaffolder and its tests,
- they are concrete learning models for understanding one BPMN feature at a time.

The samples are intentionally small. Most of them isolate one feature rather than trying to model a realistic end-to-end business process in full detail.

Model	Primary focus
`invoice_receipt.bpmn`	Baseline process with start, service work, review, approval or rejection, and terminal notification. This is the best first model to inspect.
`order_fulfillment.bpmn`	Legacy process sample kept as an additional reference model. It is useful for checking that generator changes are not too tightly coupled to the invoice-oriented examples.
`invoice_receipt_reminder.bpmn`	Intermediate timer catch event between normal process steps. Demonstrates delay-driven routing without a boundary event.
`invoice_message_exchange.bpmn`	Intermediate message throw and catch events. Shows explicit Kafka topic mappings for message-based collaboration.
`invoice_signal_exchange.bpmn`	Intermediate signal throw and catch events. Useful for contrasting signal semantics with the more direct message-event mapping.
`invoice_review_deadline.bpmn`	Interrupting timer boundary on a task. Demonstrates timeout handling and normal-flow suppression after deadline expiry.
`invoice_review_reminder_non_interrupting.bpmn`	Non-interrupting timer boundary on a task. Demonstrates reminder-style branching without cancelling the main task.
`invoice_processing_error.bpmn`	Interrupting error boundary on a task, driven by task failure events from the generated runtime.
`invoice_review_escalation.bpmn`	Interrupting escalation boundary on a task. Shows how escalation differs from failure as an explicit lifecycle outcome.
`invoice_call_activity.bpmn`	Call-activity request and reply mapping. Demonstrates the generated call and reply topics plus manual completion of the invoked activity.
`invoice_review_subprocess.bpmn`	Embedded subprocess with explicit entry and completion handlers. This is the basic scope-aware subprocess sample.

Model	Primary focus
`invoice_nested_subprocess.bpmn`	Nested embedded subprocesses. Useful for understanding recursive scope generation and subprocess-specific input and output channels.
`invoice_subprocess_deadline.bpmn`	Interrupting timer boundary attached to a subprocess scope. Demonstrates cancellation at the subprocess level rather than at one task.
`invoice_subprocess_reminder_non_interrupting.bpmn`	Non-interrupting timer boundary attached to a subprocess scope. Shows branch-without-cancel behavior for a whole scope.
`invoice_subprocess_error.bpmn`	Interrupting error boundary attached to a subprocess scope. Demonstrates subprocess-level failure propagation.
`invoice_event_subprocess_message.bpmn`	Non-interrupting embedded event subprocess with message start. Shows side-flow activation within an enclosing subprocess scope.
`invoice_event_subprocess_interrupting_message.bpmn`	Interrupting embedded event subprocess with message start. Demonstrates cancellation of the enclosing scope before the event subprocess branch starts.
`invoice_event_subprocess_timer.bpmn`	Embedded event subprocess with timer start. Demonstrates scoped delayed activation tied to the enclosing subprocess lifecycle.
`invoice_event_subprocess_error.bpmn`	Embedded event subprocess with error start. Demonstrates start-on-failure semantics driven by the canonical lifecycle stream.
`invoice_event_subprocess_escalation.bpmn`	Embedded event subprocess with escalation start. Demonstrates start-on-escalation semantics within a subprocess scope.
`data_pipeline_demo.bpmn`	Minimal data pipeline with three plugin-annotated tasks, BPMN data objects, and S3/Neo4j/PostgreSQL data stores. Demonstrates plugin execution plus data-asset metadata and data-store connector skeleton generation.
`order_events_pipeline.bpmn`	Realistic order event pipeline with 8 plugin tasks (transform, coerce, filter, enrich, validate, UUID-inject, timestamp-normalize, mask) and an XOR gateway routing by amount using runtime-evaluated conditions. Designed to be used with `-connect` for source/sink generation.
`log_processing_pipeline.bpmn`	Log processing pipeline demonstrating regex extraction, type coercion, string templating, JSON flattening, schema validation, and explicit field masking in a single linear flow. Also designed for `-connect` source/sink integration.

Model	Primary focus
`custom_activity_demo.bpmn`	Custom activity round-trip example. A ServiceTask annotated with `camunda:plugin="custom"` triggers contract interface generation and delegating worker creation for hand-written implementations.
`e2e_pipeline.bpmn`	End-to-end pipeline fixture combining category-specific plugin tasks (`json-transform`, `type-coercer`, `field-router`, `kv-enricher`, `json-schema-validator`, `window-counter`) with an XOR gateway on the `amount` field and embedded alarm configurations. Drives the end-to-end integration test.
`svenskt_aterkop.bpmn`	Non-ASCII naming coverage. Uses a Swedish process id (`återköp`) and task names (`Registrera ärende`, `Bedöm återköp`); verifies that transliteration produces valid ASCII topics and Java class names (Section 19.3).

Taken together, the sample set functions as a compact executable coverage matrix. When a new BPMN function is added to the generator, adding a dedicated sample model alongside it has become the preferred way to document the feature, test the generator, and provide a runnable teaching example.

27 Example Walkthrough

27.1 Model

The reference model is `durga-tools/src/test/resources/bpmn/invoice_receipt.bpmn`: start, service work, review, approve/reject, notify.

27.2 Generation

```
./durga durga-tools/src/test/resources/bpmn/invoice_receipt.bpmn
```

Or directly:

```
mvn -q clean package
java -jar durga-tools/target/durga-tools-0.1.0-beta.1.jar \
  durga-tools/src/test/resources/bpmn/invoice_receipt.bpmn
```

Output lands in `generated/` with Java sources, `pom.xml`, Kafka topic declarations, helper scripts, and a generated `README.md`.

27.3 Generated contents

- worker services for automatic task execution,
- waiting-state handlers for human-completion steps,
- a starter class that publishes the first process event,
- a process orchestrator that detects terminal completions,
- helper publishers and watchers for demos and testing,
- Kafka bindings in `application.yml`.

27.4 Runtime flow

1. the starter publishes the initial process event,
2. task workers consume input topics,
3. each step emits lifecycle events to `process-events`,
4. routing services move the process to the next task or end event,
5. the orchestrator emits `PROCESS_COMPLETED`.

Richer models add boundaries, subprocess scopes, and event subprocesses but follow the same pattern: handlers move through Kafka topics while `process-events` carries the lifecycle truth.

27.5 Exploring the generated process

- `demo-scenario.sh` exercises a whole process path,

- `send-task-input.sh` pushes a specific task input,
- `complete-task.sh`, `fail-task.sh`, `escalate-task.sh` force specific task outcomes,
- `watch-process-events.sh`, `watch-task-output.sh` expose the runtime flow.

28 Current Status

28.1 Maturity

Active prototype with beta-oriented hardening work underway. The generator covers the BPMN execution subset described in the execution model chapter and is regression-tested against the BPMN fixtures under `durga-tools/src/test/resources/bpmn/`. Generated projects compile and run on Quarkus + Kafka, but production adoption still requires the operational review described in the deployment, supervision and configuration chapters.

28.2 Solid

- generator refactored from monolith to multiple templates and helper classes,
- target-parameterised code generation: template groups split into `templates-java/` (the mature Java target) and `templates-rust/` (in progress), with a native Rust plugin runtime crate (`durga-rust`) whose `Plugin` contract and `ProcessEvent` wire format match the Java side (see Section 18),
- dry-run and compile-level tests cover representative BPMN features,
- core runtime contracts are unit-tested,
- monitoring topology with global-table query model,
- three-layer alarm model (AUTOMATIC / OPT_IN / EXPLICIT) with `AlarmOrigin` provenance; the monitor ships built-in STUCK and CASCADE syndromes that detect stall cascades with no per-process configuration,
- seven alarm syndromes: `HARD_ERROR`, `COUNTED`, `SLIDING_WINDOW` (event-driven), `STUCK`, `CASCADE` (absence-of-progress, evaluated by wall-clock punctuator), plus performance-SLA syndromes `SLA_LATENCY` (wall-clock ceiling per task or process) and `SLA_THROUGHPUT` (minimum completions per window), declared in the BPMN model,
- queryable alarm-state read model and dashboard alarm overlays; the dashboard surfaces system-wide alarms separately from per-process alarms,
- category-aware plugin output disposition (`PluginResult. OutputDisposition`) so routing and inspection plugins no longer overwrite the business payload,
- brace-aware inline config parsing in `KvEnricher` and `pluginConfig` escaping in the scaffolder for safe config injection,
- lifecycle-aware state counts: terminal instances (completed, failed, cancelled) bucket by outcome instead of their last activity, and aggregate-absorbed instances reach a terminal completed state,
- interrupting boundary cancellation via in-memory registry,
- GitHub Actions CI on push/PR (JDK 25 plus monitoring UI test/build gate),
- `SendTask`, `ReceiveTask`, `ScriptTask`, `BusinessRuleTask` generate dedicated handlers,
- inclusive gateway (OR) split/join,
- multi-instance loop characteristics detected and reported,
- BPMN data objects, data stores and data associations collected as pipeline metadata,
- generated `DataHandle` runtime contract for carrying large data references through process payloads,
- experimental object-store collector/extractor plugins for turning payload bytes into `DataHandle` references and resolving them again,

- handle-aware plugin execution modes for manual handle processing and materialized object-store processing,
- generated Vannak-compatible `DataIndividualMetadataEvent` emission from plugin and custom workers,
- `-connect` can generate data-store-specific Kafka Connect skeletons for BPMN data-store edges,
- 18 data pipeline catalog descriptors (16 processing plugins plus 2 connect generator descriptors) with interface-based dispatch,
- recursive descent expression evaluator for XOR and OR gateway conditions (==, !=, >, <, >=, <=, &&, ||, !, parentheses, nested field access),
- custom activity round-trip: contract interface generation, delegating workers with CDI injection, model enrichment utility,
- validation mode (Section 9): opt-in `-validation` shadow workers on both the Java and Rust targets run a candidate task against real input on a dedicated consumer index with side effects suppressed, and the monitor's `ValidationTopology` compares candidate output against production per task, exposed via `/api/validation/*`, the dashboard validation report, and `durga_validation_*` metrics.

28.3 Limits

- BPMN subset (no complex gateways; data objects are metadata, not executable runtime nodes),
- advanced scope semantics approximated,
- event subprocess semantics scoped, not generalized to all top-level cases,
- history retention is intentionally coarse,
- production deployment requires site-specific review of topic retention, replication, authentication, secrets, resource limits and alerting.

28.4 Open decisions

Top-level event subprocesses Timer, error, and escalation starts are out of the beta support boundary unless promoted with dedicated runtime semantics and integration coverage.

Runtime fidelity Whether to deepen true BPMN scope semantics as the project evolves.

Multi-instance runtime Whether detected multi-instance loop characteristics should be promoted from summary metadata to generated runtime handlers after beta.

Plugin SDK Whether to formalize a plugin development kit for third-party plugin authors beyond the current `Plugin` interface contract.

29 Glossary

BPMN Business Process Model and Notation, the process-modeling notation used as input to Durga.

Canonical lifecycle stream The shared `process-events` topic that carries normalized process and activity events.

Call activity A BPMN element used to model invocation of another process-like unit, mapped in Durga through explicit call and reply topics.

Event subprocess A subprocess triggered by an event within an enclosing scope instead of by the main sequence flow.

Execution topics The generated Kafka topics that move work between tasks, gateways and handlers.

Generated helper scripts The scaffolded scripts used to run demos, inject inputs, complete tasks, force failures and observe output.

Kafka Streams state store The local materialized store used by the monitoring app to answer low-latency queries.

Process orchestrator The generated runtime class that detects terminal flows and emits explicit completion for the enclosing process.

ProcessStateView The repository's materialized per-instance monitoring projection.

Scope cancellation The generated runtime behavior that suppresses or redirects normal flow after an interrupting boundary or interrupting event subprocess fires.

Validation mode An opt-in mode (`-validation`) in which a candidate task implementation is run against real input on a dedicated consumer index with side effects suppressed, and its output is compared per task against the prior/production output (Section 9).

Shadow worker The generated validation-mode worker that consumes production input through a dedicated consumer group, runs the candidate implementation without side effects, and diverts its output to `validation-candidate-outputs` instead of the task output topic.

Candidate output A `ValidationCandidateOutput` record emitted by a shadow worker, carrying the shared input and the candidate's output for comparison against production.

Validation result A `ValidationResult` record classifying one comparison as `EQUAL`, `DIFF`, `PRIOR_MISSING`, or `CANDIDATE_ERROR`, with the located differences.

Scaffolder The BPMN-driven generator entry point, implemented by:

```
org.gautelis.durga.tools.BpmnScaffolder
```

Subprocess scope service A generated class that manages entry, completion and scope-level lifecycle behavior for an embedded subprocess.

1 Integration test catalogue

This appendix documents every test suite in the project that exercises behaviour against a live Kafka broker (Docker-backed integration tests) plus the fault detection and alarm-state unit test suites for completeness. Docker-backed integration tests extend `KafkaIntegrationTestBase`, which uses Testcontainers to spin up a real Kafka broker (Confluent CP 7.8.0 in KRaft mode) per test class. Each integration test class creates its own topics with randomised suffixes to avoid collisions and disposes of all resources in `@After` / `@AfterClass` hooks.

The integration tests are run with:

```
mvn test -Dtest='*IntegrationTest'
```

Timeouts default to 60 seconds and can be overridden with `-Ddurga.it.timeout.seconds=N` or the environment variable `DURGA_IT_TIMEOUT_SECONDS`.

1.1 ProcessEventKafkaIntegrationTest

Purpose: Verify that `ProcessEvent` records survive a Kafka produce-consume round-trip with full wire-format fidelity.

Topic: A single non-compacted topic created per test class. The test uses a plain `KafkaProducer` and `KafkaConsumer` with manual offset management (`enable.auto.commit=false`, `auto.offset.reset=earliest`).

Scenarios (3):

shouldProduceAndConsumeSingleEvent Publishes one `ProcessEvent` with an `ErrorInfo` payload (`code="TASK_FAILED"`). Consumes it and asserts that the `processInstanceId`, `processId`, `eventType` and `error().code()` are preserved exactly. Exercises the full `toJson` / `fromJson` pipeline over the wire.

shouldProduceAndConsumeMultipleEvents Publishes five distinct events with monotonically increasing `processInstanceId`s, consumes them all in an at-least-once poll, collects IDs, and asserts all five were received and each has a valid `processId` and `eventType`.

shouldRoundTripFromJsonToJson Publishes a compact-constructor event (no explicit `eventType`, relying on the `Status` inference), consumes it, re-serializes the parsed result, parses again, and asserts that the double-parsed event matches the original `processInstanceId`, inferred `eventType`, and nested `payload` values.

Why it matters: The entire monitoring model depends on `ProcessEvent` JSON being correctly serialised by producers and correctly parsed by monitoring topology consumers. Any mismatch would silently corrupt state stores.

1.2 ProcessStateStoreIntegrationTest

Purpose: Verify that the compacted-topic `ProcessStateStore` persists and retrieves `ProcessState` records with correct latest-wins semantics.

Topic: A single compacted topic (`cleanup.policy=compact`). The store is a thin wrapper around a `KafkaProducer` (for writes) and a `KafkaConsumer` (for reads on a compacted topic).

Scenarios (5):

shouldSaveAndLoadLatest Saves one `ProcessState` with a known `processInstanceId`, then loads it back. Asserts the returned state has the same instance ID, version, variables, and token list.
shouldReturnLatestAfterMultipleSaves Saves three states for the same key with incrementing version numbers (`ver=1,2,3`). The load must return version 3.
shouldHandleMultipleKeys Saves three distinct keys (`pi-a`, `pi-b`, `pi-c`) and verifies each is independently retrievable with the correct variables.
shouldReturnNullWhenKeyNotFound Calls `loadLatest` on a non-existent key and asserts `null` is returned within the timeout.
shouldHandleTimeoutGracefully Calls `loadLatest` with a zero duration timeout and verifies `null` is returned cleanly (no exception).

Why it matters: The `ProcessStateStore` is used by generated projects to snapshot process state to a compacted topic. Correct latest-read semantics are critical for crash recovery.

1.3 MonitoringTopologyIntegrationTest

Purpose: End-to-end verification of the Kafka Streams monitoring topology against a real broker.

Topics: Seven topics (events, state, counts, active, latency, trends, models) created with UUID suffixes. The topology uses `process-events-.*` regex subscription to consume from the events topic and materialises results into state stores queried through `ProcessMonitoringQueryService`.

Scenarios (5):

shouldMaterializeProcessStateFromLifecycleEvents Publishes three events (`PROCESS_STARTED`, `ACTIVITY_ENTERED`, `ACTIVITY_COMPLETED`) for one instance, waits for state materialisation, and asserts the lifecycle is `"active"` with the correct `currentActivityId` and a populated `activityDurationsMs` map.
shouldProduceStateCounts Publishes events for two instances of the same process, then queries the count store. Asserts at least one instance appears in the count aggregation.
shouldProduceLatencySummaries Publishes an `ACTIVITY_ENTERED` followed 4 minutes later by `ACTIVITY_COMPLETED` for the same activity. Queries the latency store and asserts the returned `ActivityLatencySummary` has a `sampleCount` ≥ 1 and `maxDurationMs` $\geq 240\,000$ ms. This validates the entry-store-based latency computation pipeline.
shouldDetectStuckInstances Publishes an `ACTIVITY_ENTERED` that is never completed, then queries `stuckInstances()` with a 1-second threshold. Asserts the instance appears as stuck with `lifecycleState "active"`.
shouldMaterializeProcessStateFromUnkeyedLifecycleEvents Publishes a `PROCESS_STARTED` event with a `null` Kafka message key. Asserts that the monitoring topology extracts the `processInstanceId` from the value payload and materialises state correctly. This verifies the unkeyed-event fallback path in the topology.

Why it matters: This is the most important integration test suite. It validates the monitoring topology's ability to materialise state, compute latency percentiles, detect stuck instances, and handle edge cases like unkeyed events — all over a real Kafka broker.

1.4 ChaosIntegrationTest

Purpose: Resilience verification of the monitoring topology under crash-and-restart cycles.

Infrastructure: Same topic set as `MonitoringTopologyIntegrationTest`. Uses a fixed `application.id` across restarts so that Kafka Streams rebalances and replays from the changelog topics.

Scenarios (3):

shouldRecoverStateAfterTopologyRestart Publishes a complete task flow (`PROCESS_STARTED` → `ACTIVITY_ENTERED` → `ACTIVITY_COMPLETED` → `PROCESS_COMPLETED`), waits for the state to reach `"completed"`, kills the topology, restarts it, waits for re-materialisation, and asserts the recovered state matches the pre-crash state (same `processInstanceId`, `processId`, `lifecycleState`).
shouldNotDuplicateStateEntriesAfterRestart Publishes a complete flow, records the count-store total, kills and restarts the topology, and asserts the count-store total is unchanged — no spurious duplicate events are generated by the rebalance.
shouldHandleMultipleRestarts Publishes data once, then cycles the topology through three start-kill-restart iterations (0, 1, 2 restarts), asserting at each iteration that the original state is still present with the correct `lifecycleState` and `processInstanceId`.

Why it matters: Durga is designed for continuous operations; topology restarts happen during deployments, scaling events, and failure recovery. These tests verify that the state-store changelogs produce correctly idempotent results.

1.5 E2EPipelineIntegrationTest

Purpose: Full data pipeline end-to-end verification combining scaffolder output validation with monitoring topology event handling for all category-specific lifecycle events introduced in the current release.

Fixture: `durga-tools/src/test/resources/bpmn/e2e_pipeline.bpmn` — a pipeline exercising `json-transform`, `type-coercer`, `field-router` (route), `kv-enricher` (enrich), `json-schema-validator` (validate), and `window-counter` (aggregate), with an XOR gateway branching on the `amount` field.

Scenarios (6):

shouldGeneratePluginExecutorsWithCategorySpecificEvents Runs the scaffolder on the E2E fixture and inspects the generated executor classes:

- `RouteDecisionPluginExecutor.java` must contain `ProcessEvent.EventType.GATEWAY_TAKEN` (route category).
- `ValidateHighValuePluginExecutor.java` must contain `ProcessEvent.EventType.ACTIVITY_ESCALATED`, `ProcessEvent.Status.ESCALATED`, and error code `"VALIDATION_FAILED"` (validate category).
- `AggregateHighValuePluginExecutor.java` must contain `ProcessEvent.EventType.ACTIVITY_ENTERED` (aggregate category, null-output path).

shouldHandleGatewayTakenEvents Publishes `PROCESS_STARTED` → `ACTIVITY_ENTERED` → `GATEWAY_TAKEN` for `route_decision` and verifies the monitoring state view records a duration for the routed activity with lifecycle `"active"`.
shouldHandleActivityEscalatedEvents Publishes `PROCESS_STARTED` → `ACTIVITY_ENTERED` → `ACTIVITY_ESCALATED` for `validate_high_value` and verifies the monitoring topology records

the escalated activity's duration correctly.

shouldHandleActivityEnteredThenCompletedEvents Publishes `PROCESS_STARTED` → `ACTIVITY_ENTERED` → `ACTIVITY_COMPLETED` for `aggregate_high_value` (30 s gap) and asserts that the latency query returns an `ActivityLatencySummary` with ≥ 1 sample and `maxDurationMs` $\geq 30\,000$ ms. This validates the entry-store latency pipeline for aggregate window-start events.

shouldHandleMultipleEventTypesInLifecycle Simulates a full pipeline trace across five activities with mixed event types:

- `transform_order`: entered → completed
- `coerce_types`: entered → completed
- `route_decision`: entered → `GATEWAY_TAKEN`
- `enrich_high_value`: entered → completed
- `validate_high_value`: entered → `ACTIVITY_ESCALATED`

Asserts all five activities appear in the state view's duration map and the lifecycle state is `"active"`. Each test method uses an isolated `processId` to prevent cross-test contamination in the shared latency store.

shouldParseAlarmConfigsFromBpmnModel Parses the E2E fixture with `BpmnAlarmConfigParser` and asserts that alarm configurations are present at all three levels: an activity-level `HARD_ERROR` alarm on `validate_high_value`, a process-level inherited `COUNTED` alarm (threshold 5) applied to every activity, and a process-level aggregate `SLIDING_WINDOW` alarm (threshold 3, 60 s window).

Why it matters: This test closes the gap between the scaffolder's code-generation contract and the monitoring topology's runtime behaviour. It proves that the category-specific event types (`GATEWAY_TAKEN`, `ACTIVITY_ESCALATED`, `ACTIVITY_ENTERED`) are correctly generated by the scaffolder AND correctly consumed and materialised by the monitoring engine — end to end, against a live Kafka broker.

1.6 ValidationTopologyIntegrationTest

Purpose: Verifies the validation (shadow-run) topology end to end against a live Kafka broker: candidate-task outputs published to the validation-candidate topic are joined against production outputs and the comparison outcome is materialised into the validation results store.

Scenarios (6):

shouldMarkEqualWhenCandidateMatchesPriorRegardlessOfArrivalOrder A candidate output that matches the production output resolves to `EQUAL`, even when the candidate arrives before the prior output (the join is order-independent).

shouldMarkDiffWithLocatedDifference A differing candidate output is recorded as `DIFF` together with the located difference.

shouldMarkPriorMissingWhenNoProductionOutput A candidate with no corresponding production output is recorded as `PRIOR_MISSING`.

shouldMarkCandidateError A candidate that reports an execution error is recorded as `CANDIDATE_ERROR` even when a prior output exists.

shouldIgnoreConfiguredPaths Differences under a configured ignore path (e.g. volatile fields) do not count as a divergence.

shouldSummarizePerTask The per-task validation summary correctly counts `EQUAL` and `DIFF` outcomes.

Why it matters: Validation mode lets a candidate task be shadow-run against production without affecting the live pipeline; this suite proves the comparison outcomes are computed and materialised correctly over

real Kafka.

1.7 SlaMonitoringIntegrationTest

Purpose: End-to-end verification of the performance-SLA alarm path. Runs the fault-detection topology (which measures SLAs from lifecycle events and emits alarms to `fault-alarms`) together with the alarm-state topology (which folds those alarms into the queryable read model consumed by `/api/metrics`), both against a live Kafka broker.

Scenarios (2):

shouldFireSlaLatencyAlarmWhenTaskExceedsLimit Publishes `ACTIVITY_ENTERED` then `ACTIVITY_COMPLET` four minutes apart for a task with a 1 s `SLA_LATENCY` ceiling. Asserts the alarm-state store materialises an `SLA_LATENCY` alarm whose observed latency (`lastCount`) is 240 000 ms and limit (`threshold`) is 1 000 ms — the exact values surfaced as `durga_sla_last_observed` / `durga_sla_limit`.
shouldFireSlaThroughputAlarmWhenRateStarves Declares a process-wide `SLA_THROUGHPUT` floor of two completions per two-second window, feeds a single completion, and asserts the wall-clock punctuator reports a breach (`lastCount` below `threshold`) once a full window has elapsed with insufficient throughput. Uses a 1 s scan interval override for prompt scoring.

Why it matters: Proves that SLAs declared in the process model turn into real, queryable alarms driven by observed wall-clock latency and throughput — not merely a static counter.

1.8 FaultDetectionTest (unit)

Purpose: Unit-test the fault detection algorithm across all three syndromes without requiring a Kafka broker. The test uses in-memory `MapKVStore` adapters that implement the `KeyValueStore` interface.

Class: `FaultDetectionTest` (14 scenarios, no Docker required).

Scenarios (14):

shouldFireHardErrorOnFirstOccurrence Config with scope (`processId="order-proc"`, `activityId="validate_order"`), `HARD_ERROR` syndrome, `CRITICAL` severity. Asserts alarm fires with correct message rendering, scope fields, and count=1.
shouldFireHardErrorRepeatedly Wildcard-scoped config (both `processId` and `activityId` null) fires on every matching `PROCESS_FAILED` event regardless of process or activity.
shouldNotFireHardErrorForNonMatchingEventType Config targeting `ACTIVITY_ESCALATED` ignores a `PROCESS_FAILED` event.
shouldMatchByProcessOnly Config scoped to `"order-proc"` fires for events in that process, ignores events in `"other-proc"`.
shouldMatchByActivityOnly Config scoped to `"validate_order"` fires for that activity in any process, ignores other activities.
shouldCountAndFireWhenThresholdExceeded `COUNTED` syndrome with threshold=3. First three events increment count silently; fourth crosses threshold and fires alarm; fifth is suppressed (single-fire).
shouldReArmAfterCountDropsBelowThreshold After an alarm fires, manually resets count below threshold and clears the alarm marker. The next event that crosses threshold fires a new alarm.
shouldFireWhenWindowCountExceedsThreshold `SLIDING_WINDOW` with threshold=2 and 60-second window. First three events accumulate; third crosses threshold and fires; fourth suppressed (single-fire).

shouldPruneOldTimestampsInWindow With a 100-ms window, inserts four events, waits 150 ms, then adds a fifth. The four old timestamps are pruned; only the one new event remains in the window, so no alarm fires.
shouldHandleMultipleConfigs A `HARD_ERROR` and a `COUNTED` config both match the same event. The hard-error fires immediately; the counted config correctly matches the event's scope and type.
shouldHandleEscalatedEvents Config targeting `ACTIVITY_ESCALATED` fires on an escalated event with correct syndrome and config ID.
shouldBuildTopologyWithRegexEventsAndModelRegistry Verifies that the fault detection topology subscribes to lifecycle event topics by regex and includes the process-model registry input needed for BPMN alarm configuration.
shouldRejectSlidingWindowWithoutDuration Constructor validation rejects a `SLIDING_WINDOW` config with `null windowDuration`.
shouldRejectCountedWithoutThreshold Constructor validation rejects a `COUNTED` config with `threshold=0`.

Why it matters: The fault detection algorithm is the core intelligence that distinguishes between hard errors, counted thresholds, and time-windowed bursts. Getting the de-duplication and re-arm logic wrong would cause either alarm floods or silent failures.

1.9 AlarmStateViewTest and AlarmStateTopologyTest (unit)

Purpose: Unit-test the alarm-state read model that folds emitted `AlarmEvent` records into queryable `AlarmStateView` records for the dashboard and HTTP API.

Classes: `AlarmStateViewTest` (4 scenarios) and `AlarmStateTopologyTest` (1 scenario), no Docker required.

Scenarios (5):

shouldBuildStateFromAlarmEvent Converts a single `AlarmEvent` into an `ACTIVE` state view with the expected key, timestamps, fire count, and message.
shouldMergeRepeatedAlarmEventsAndKeepLatestDetails Folds repeated firings for the same process/activity/config into one view, increments `fireCount`, keeps the strongest severity, and records the latest firing details.
shouldNotLetLateOlderEventOverwriteLatestDetails Ensures an older event that arrives late does not overwrite the newest alarm ID, instance ID, message, or count.
shouldUseWildcardActivityForProcessWideAlarms Verifies that process-wide alarms use `*` as the activity component in the alarm-state key.
shouldBuildAlarmStateStoreTopology Verifies that the alarm-state topology consumes `fault-alarms` and materializes `alarm-state-store`.

Why it matters: Fault detection emits a stream of alarm firings, but operators need a stable read model. These tests protect the fold-up semantics used by `/api/alarms`, per-process alarm queries, instance alarm details, and dashboard overlays.

1.10 Infrastructure

The Docker-backed suites share a common base class:

KafkaIntegrationTestBase (`durga-runtime/src/test/java/org/gautelis/durga/KafkaIntegrati java`) uses the Testcontainers library to start a single Confluent CP Kafka 7.8.0 broker in KRaft mode (no ZooKeeper). The broker listens on a random free host port and advertises `localhost:<port>`. Docker availability is auto-detected; if Docker is missing, tests are skipped unless `DURGA_REQUIRE_DOCKER_TESTS=true` is set.

- **Environment variables:** `DURGA_DOCKER_HOST_IP` (override advertised listener host), `DURGA_REQUIRE_DOCKER_TESTS` (fail if Docker unavailable), `DURGA_IT_TIMEOUT_SECONDS` (override 60 s default).
- **System properties:** `durga.it.timeout.seconds` (same as env var, takes precedence), `DOCKER_HOST` (Docker socket URI, auto-detected if unset).

Each integration test class creates topics with random UUID suffixes in its `@BeforeClass` and tears down the broker in `@AfterClass`. Test methods create their own `KafkaProducer`, `KafkaConsumer`, or `KafkaStreams` instances in `@Before` and close them in `@After`.

- **Run all integration tests:** `mvn test -Dtest='*IntegrationTest'`
- **Run a specific suite:** `mvn test -Dtest='E2EPipelineIntegrationTest'`
- **Require Docker (fail if unavailable):** `mvn test -Ddurga.integration.requireDocker=true -Dtest='*IntegrationTest'`